

2. НАСТРОЙКА РАБОЧЕГО ОКРУЖЕНИЯ

В ходе прохождения учебной практики использовался язык программирования Python, среда разработки (IDE) VS Code (рисунок 1-2)



Рисунок 1 – иконка приложения VS Code

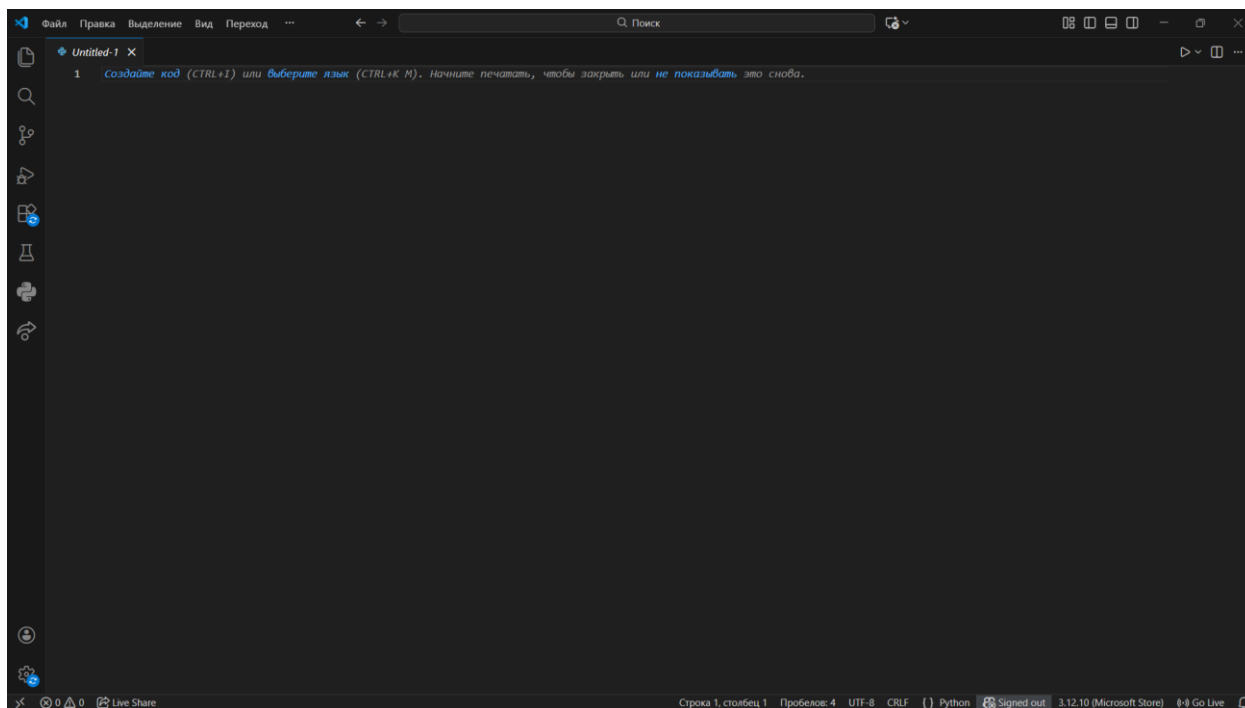


Рисунок 2 – интерфейс VS Code

Python – это простой и популярный язык программирования, на котором создают самые разные программы: сайты, ботов, приложения, системы анализа данных и даже искусственный интеллект. Его часто выбирают новички, потому что код на нём выглядит понятно и читается почти как обычный текст, без сложных символов и громоздких конструкций. На Python можно быстро написать рабочую программу и проверить свою идею, не тратя

много времени на сложные настройки. У языка огромное количество готовых библиотек, поэтому разработчикам не нужно всё делать с нуля — для многих задач уже есть инструменты. Python работает на разных операционных системах и используется как в обучении, так и в крупных компаниях. При этом он не самый быстрый по скорости работы и не всегда подходит для очень тяжёлых или требовательных к производительности задач, например, для создания сложных игр или систем реального времени. Тем не менее, благодаря простоте, универсальности и большому сообществу, Python остаётся одним из самых удобных и востребованных языков программирования в мире.

Плюсы Python для создания игры «Длинные нарды»

– Простота описания правил игры – Python имеет понятный и читаемый синтаксис, поэтому правила нард (ходы, выбивание, ограничения на передвижение, проверка допустимости хода) можно реализовать в виде логических условий без перегруженного кода. Это особенно важно для игры, где много исключений и проверок.

– Удобная работа со структурами данных (доска и фишки) – Игровое поле легко представить в виде:

- списка списков,
- словаря позиций,
- массива объектов. Каждую лунку можно хранить как ячейку списка, а фишки — как числа или объекты. Перемещение фишек сводится к изменению значений в структуре данных — без сложной математики или низкоуровневых операций.

– Встроенная работа со случайными числами (кубики) – Для броска кубиков используется стандартный модуль «random». Не нужно подключать внешние инструменты – генерация случайных чисел уже есть в языке.

– Быстрая разработка прототипа – Python отлично подходит для быстрого старта:

- мало служебного кода,
- нет сложной компиляции,

– легко тестировать по частям.

Можно сначала сделать текстовую версию игры, а потом постепенно добавить интерфейс.

– Подходит для реализации искусственного соперника – Python широко используется в алгоритмах и логике. Можно реализовать:

- простого бота (случайные ходы),
- бота с оценкой позиции,
- продвинутую стратегию на основе поиска (Minimax).

– Большое количество библиотек для графики и интерфейса – Игра не обязательно должна быть текстовой. Python позволяет сделать визуальную версию с помощью:

- Pygame — создание 2D-игр, рисование доски, анимация фишек
- Tkinter — простые окна и интерфейс
- PyQt / PySide — более профессиональный графический интерфейс
- Kivy — игры и приложения с поддержкой сенсорных экранов

– Кроссплатформенность – Игра на Python может работать на Windows, Linux и macOS без серьезных изменений.

– Удобство отладки – Ошибки легко отслеживать, код читаемый, можно быстро проверять отдельные части программы.

Минусы Python для создания игры «Длинные нарды»

- Меньшая производительность по сравнению с C++ или C#. Python – интерпретируемый язык. Если в игре будет сложная графика, анимация или вычисления, она может работать медленнее.
- Не лучший выбор для мощной 3D-графики – Для реалистичной 3D-графики чаще используют Unity (C#) или Unreal Engine (C++). Python больше подходит для 2D-игр и логических проектов.
- Интерфейс нужно реализовывать отдельно – Сам по себе Python не создает красивое окно. Нужно использовать библиотеки (Pygame, Tkinter и т.д.), что требует дополнительного изучения.
- Возможен «беспорядок» в коде – Из-за большого количества правил игры код может стать запутанным, если:
 - не разделять логику на функции,
 - не использовать классы,
 - не структурировать проект.
- Распространение игры сложнее, чем у компилируемых языков – Чтобы игра запускалась у других пользователей, может потребоваться установка Python или сборка в .exe файл.

Почему Python хорошо подходит для этой игры

Python особенно подходит для создания игры «Длинные нарды», потому что игра в основном основана на логике и правилах, а не на графике или высокой производительности. В отличие от C++, Java или Lazarus/Delphi, в Python проще и быстрее описать ходы, проверку правил и работу с доской – код получается короче и понятнее. По сравнению с JavaScript он удобнее для реализации алгоритмов и стратегии компьютерного соперника. Хотя Python медленнее и ограничен в сложной графике, для настольной логической игры

это не критично, а готовые библиотеки, такие как Pygame, Tkinter или PyQt, позволяют легко добавить интерфейс и анимацию.

Для системы контроля версий в соответствии с заданием использована система GitHub. [skree3/Long-backgammon](https://github.com/skree3/Long-backgammon) – ссылка на репозиторий. [skree3](https://github.com/skree3) – ссылка на профиль (рисунок 3)

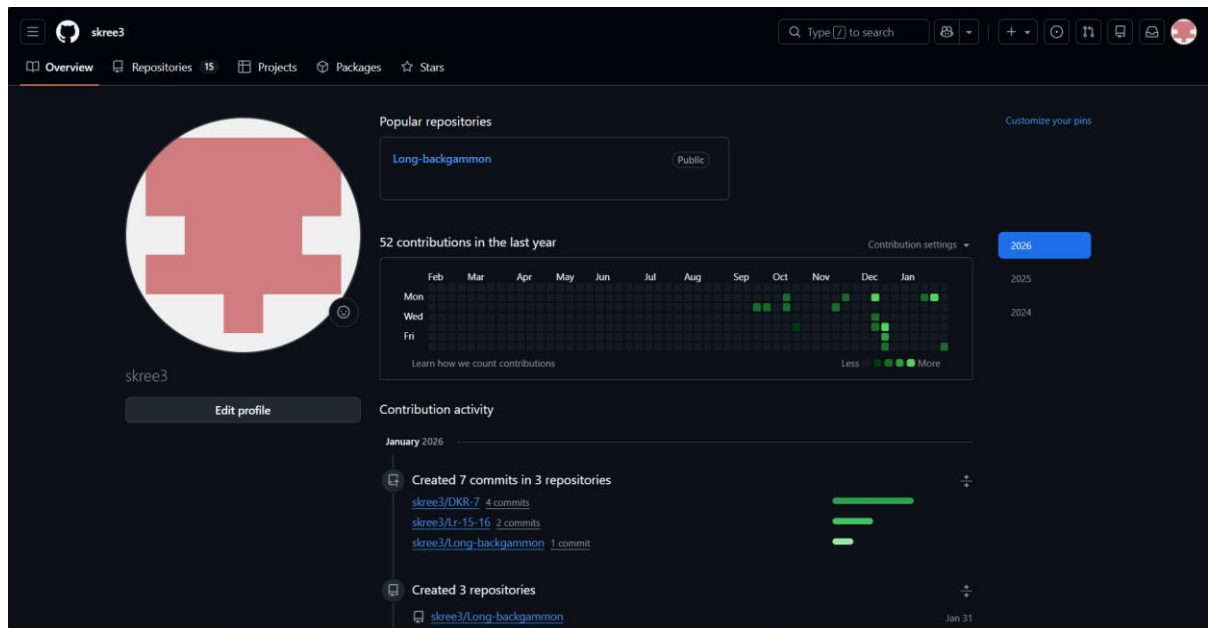


Рисунок 3 – профиль GitHub

4. ОПИСАНИЕ ВЫПОЛНЕНИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

4.1 Анализ предметной области

Нарды — игра, в которой перемещаются фишки на поле или столе, цель игры — гонка к цели, а движение фишек управляется броском двух кубиков. Элементы удачи и мастерства хорошо сбалансированы в нардах, так что каждый из них обычно необходим для победы [1]. Цель игры — провести все свои фишки по игровому полю и вывести их за пределы доски раньше соперника (вид игровой доски для нардов представлен на рисунке 1) [2]



Рисунок 1 – Игровая доска для нардов

Игровое поле состоит из 24 узких треугольных пунктов, объединённых в четыре квадранта. У каждого игрока имеется по 15 фишек, которые движутся по строго определённой траектории в противоположных направлениях. Начальная расстановка фишек показана на рисунке 2.

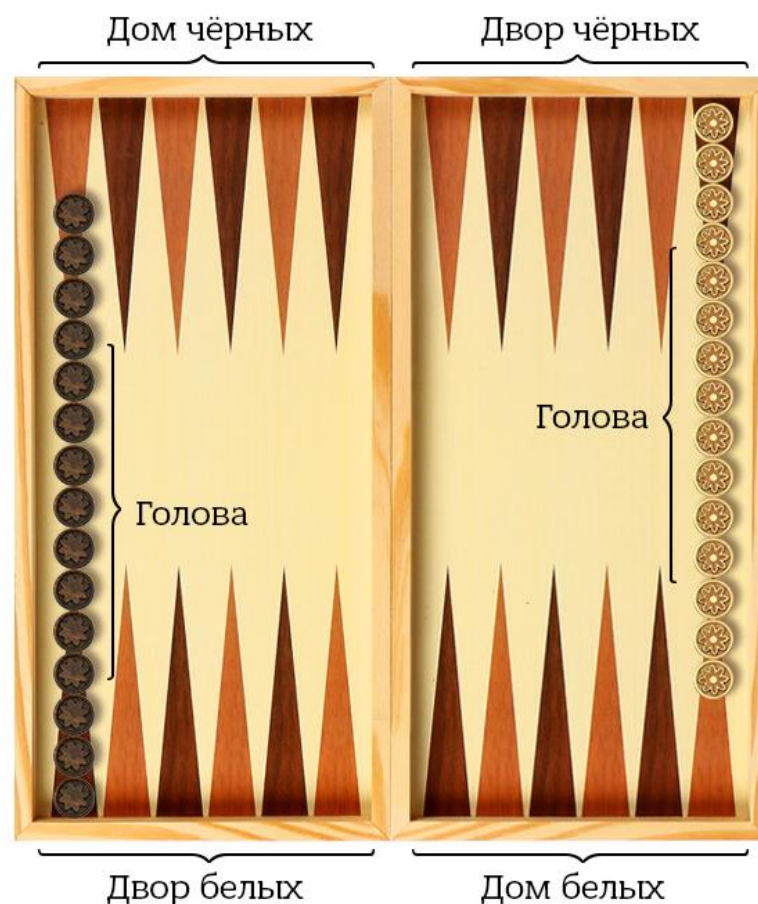


Рисунок 2 – Начальная расстановка фишек в нардах

Ключевые элементы предметной области:

- Игровое поле — это специальная доска, на которой игроки размещают фишки (шашки);
- Фишки — круглые плоские фигуры, которые используются в игре;
- Игральные кости — небольшие предметы (многогранники), используемые как инструменты для азартных игр и социальных игр. Наиболее распространённой формой штампов является куб, каждая сторона которой отмечена от одной до шести маленьких точек (пятен). Пятна расположены по обычным схемам и размещаются так, что пятна на противоположных сторонах всегда суммируются до семи (смотрите рисунок 3) [3];
- Правила перемещения — правила расстановки фишек, совершения ходов, выбрасывания фишек с игрового поля и окончания игры [1];

- Стратегия игры — набор тактик, которые помогают игрокам увеличить шансы на победу, учитывая особенности игры и ситуацию на поле.



Рисунок 3 – Игральные кости, используемые в нардах

С точки зрения моделирования нарды представляют интерес как система, сочетающая:

- детерминированные правила;
- случайные события;
- стратегическое планирование;
- анализ рисков и вероятностей.

Таким образом, нарды можно рассматривать как модель процесса принятия решений в условиях неопределённости, что делает данную предметную область актуальной для разработки интеллектуальных алгоритмов и программных игровых систем.

Игры типа нард имеют долгую историю развития. Аналоги можно разделить на исторические и современные программные.

Исторический аналог 1 — Триктрак

Триктрак — старинная французская настольная игра с XV века, где шашки по доске передвигают по числу очков, выпавших на костях. Имела восточное происхождение (из Персии). Существовало много правил и особая терминология, усложнявшая игру в триктрак.

Была известна и в древности: у греков под именем *diagrammismos* и у римлян под названием *duodena scripta* (о ней поведал Овидий). В русском языке триктрак синоним игры в нарды [4].

Триктрак является европейской игрой, которая считается прямым предшественником современных нард. Игровая доска триктрака показана на рисунке 4.



Рисунок 4 – Игровая доска для триктрака

Сходства с нардами:

- использование костей;
- движение фишек по пунктам;
- необходимость стратегии;

- вывод фишек из игры.

Отличия:

- особенности расстановки;
- различия в правилах взятия;
- иная система подсчёта очков.

Исторический аналог 2 — Царская игра Ура

Царская игра Ура — древняя настольная игра для двух игроков, появившаяся в древней Месопотамии примерно в начале III тысячелетия до н. э. Предположительно, игра пользовалась популярностью на всём Ближнем Востоке того времени среди всех слоёв населения, а её доски находят даже в достаточно удалённых от Месопотамии регионах, таких как Крит и Шри-Ланка [5]. Поле Царской игры Ура представлено на рисунке 5.



Рисунок 5 – Древняя доска Царской игры Ура

Сходства с нардами:

- пошаговое перемещение фишек;
- наличие случайности;
- соревновательный характер.

Отличия:

- форма поля;
- упрощённая структура маршрута;
- меньшая стратегическая глубина.

Общий вывод по недостаткам исторических аналогов

Проведённый анализ показал, что существующие аналоги, несмотря на их историческую, игровую и прикладную ценность, не обеспечивают комплексного подхода к изучению и моделированию игры в нарды. Исторические аналоги отражают ранние этапы развития настольных игр данного типа, однако обладают упрощённой механикой и недостаточной формализацией правил, что ограничивает их применимость в современных исследованиях. [6][7]

4.1.1 Анализ проблемы

Основной проблемой в предметной области нарда является выбор оптимального хода при наличии случайного фактора. Игрок должен учитывать:

- текущее расположение фишек (пример позиции представлен на рисунке 6);
- вероятность выпадения нужных значений на костях;
- возможные действия соперника;
- риск взятия фишки;
- баланс между атакой и защитой.

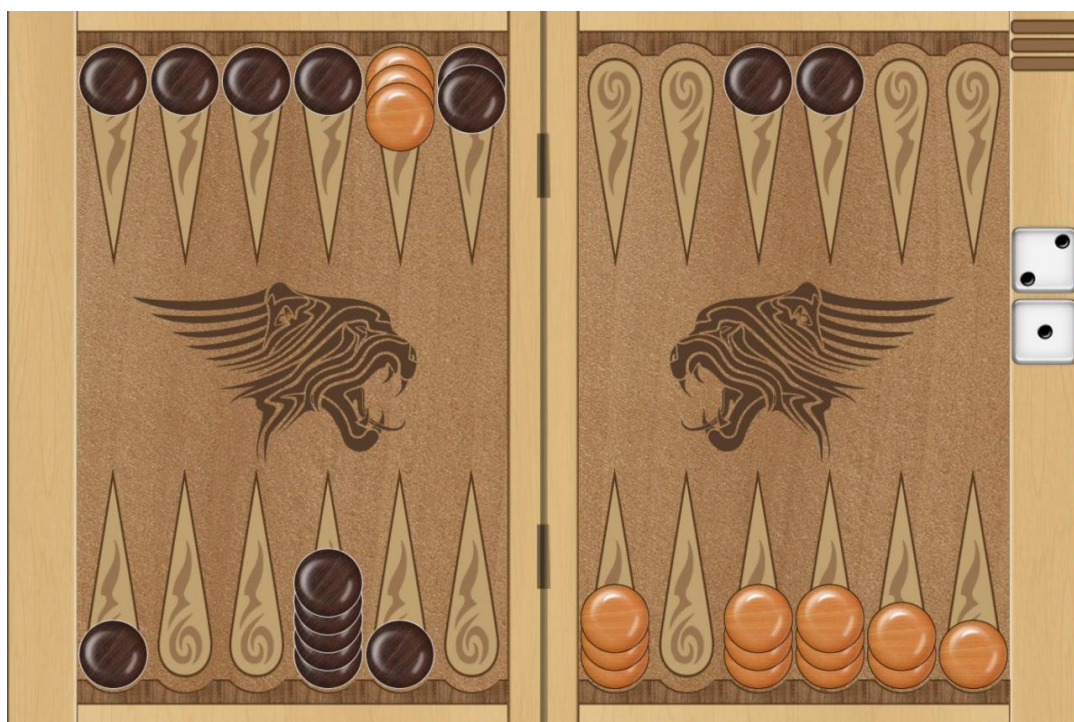


Рисунок 6 – Пример игровой позиции в процессе партии

В отличие от полностью детерминированных игр, развитие партии в нардах невозможно точно спрогнозировать. Поэтому решения принимаются на основе вероятностных оценок.

Процесс принятия решения можно представить следующим образом:

- анализ текущего состояния доски;
- определение допустимых ходов;
- оценка рисков и выгод;
- выбор оптимального варианта;
- корректировка стратегии.

4.1.2 Обзор аналогов

Онлайн аналог игры на сайте https://yandex.ru/games/app/190906?utm_source=game_promo_catalog&yclid=11995594610743967743 – Яндекс игры.

Плюсы:

- Выбор стиля игры (рисунок 7)

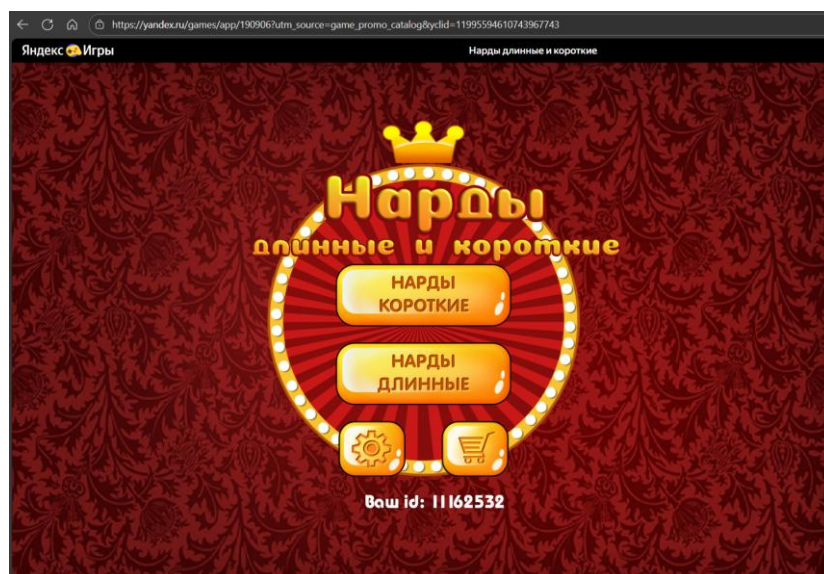


Рисунок 7 – стили игры

- Выбор соперника. Бот или человек (по интернету) (рисунок 8)

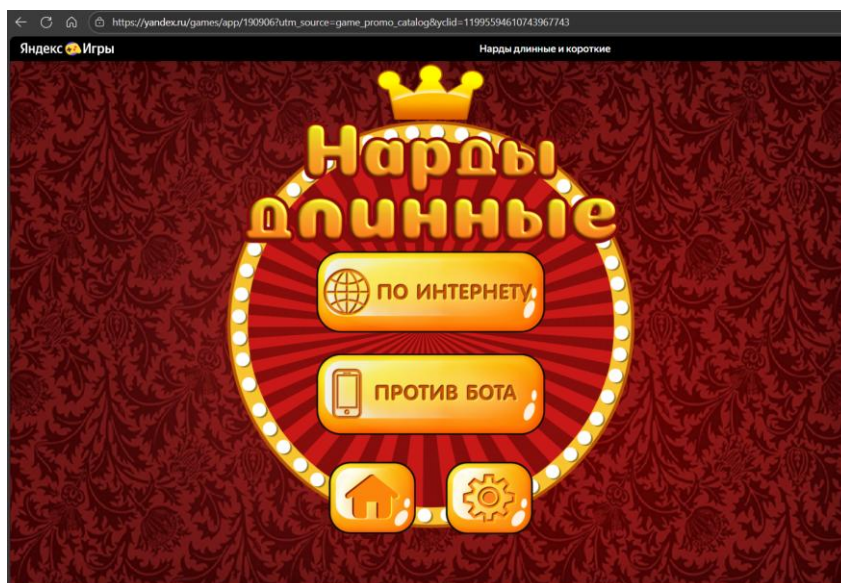


Рисунок 8 – выбор соперника

- Простой и понятный интерфейс без лишних деталей(рисунок 9)

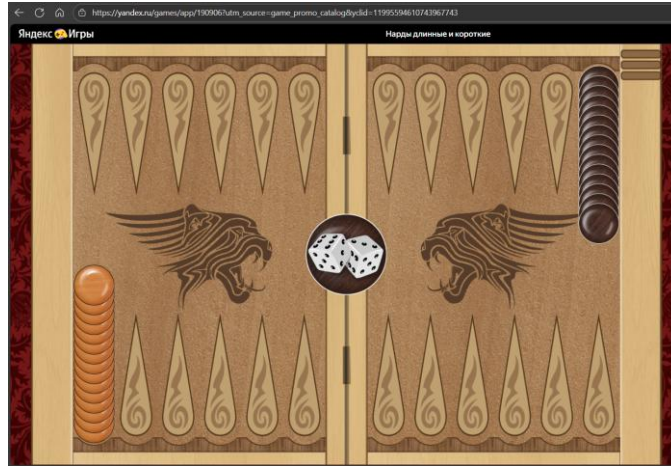


Рисунок 9 – интерфейс

Минусы:

- Постоянная реклама
- Нужен доступ к интернету
- Доступно только 75% игрового экрана (рисунок 10)

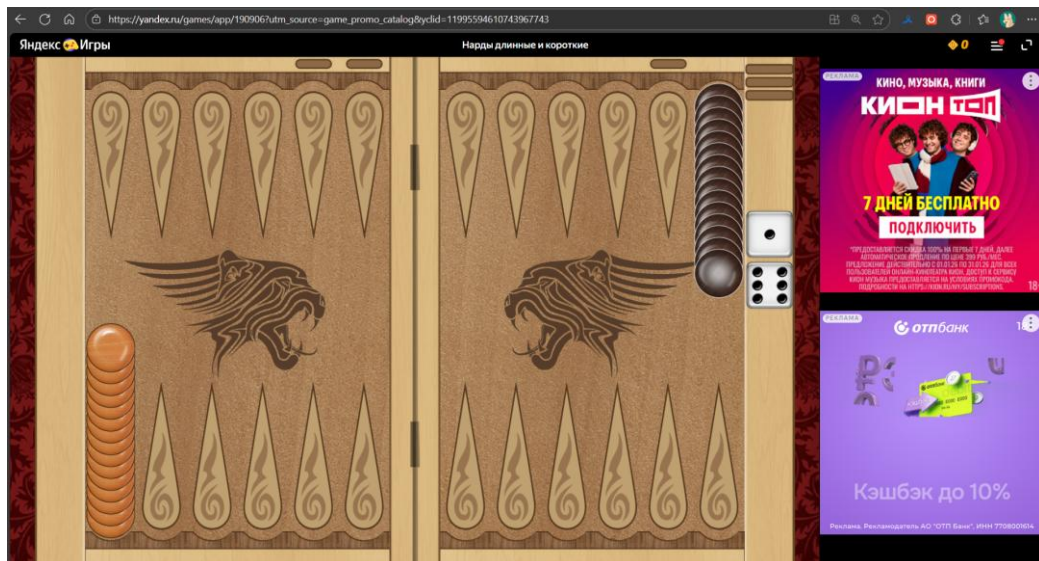


Рисунок 10 – игровой экран

Онлайн аналог игры на сайте <https://vk.com/games/nardegame?ysclid=mks8b8mc1s693221843> – вк ишры

Плюсы:

- Нельзя просто так выйти с игры (рисунок 11)

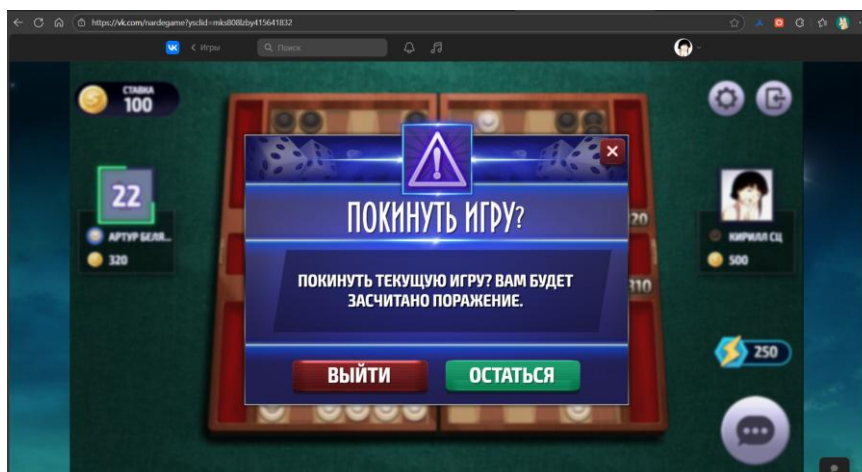


Рисунок 11 – выход с игры

- Помощь (рисунок 12)

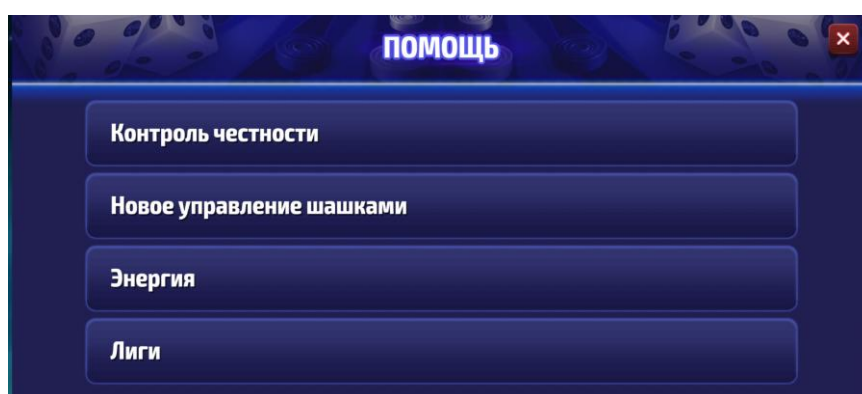


Рисунок 12 – помощь

- Можно играть в полноэкранном режиме
- Отсутствие рекламы

Минусы:

- Нельзя выбрать противника, только человек с сети (рисунок 13)

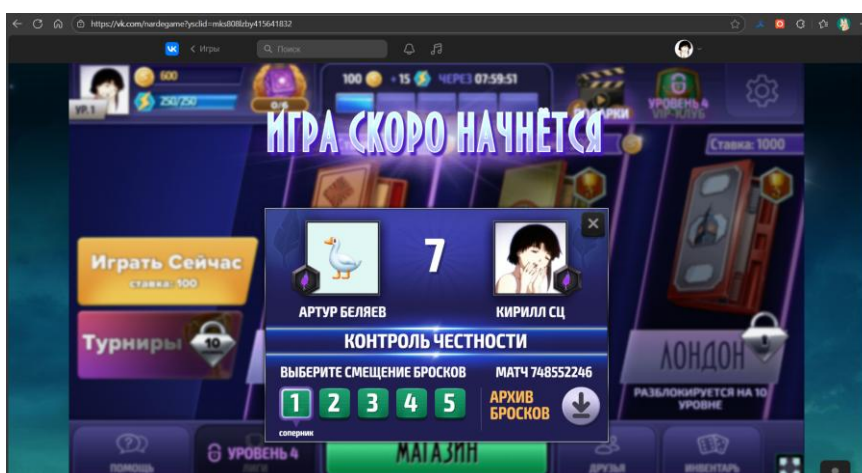


Рисунок 13 – противник

- Слишком загруженный интерфейс, много лишнего (рисунок 14)



Рисунок 14 - интерфейс

- Только с подключением интернета

Аналог игры с Google Play
<https://play.google.com/store/apps/details?id=org.aastudio.games.longnards>.

Плюсы:

- Простой интерфейс (рисунок 15-16)

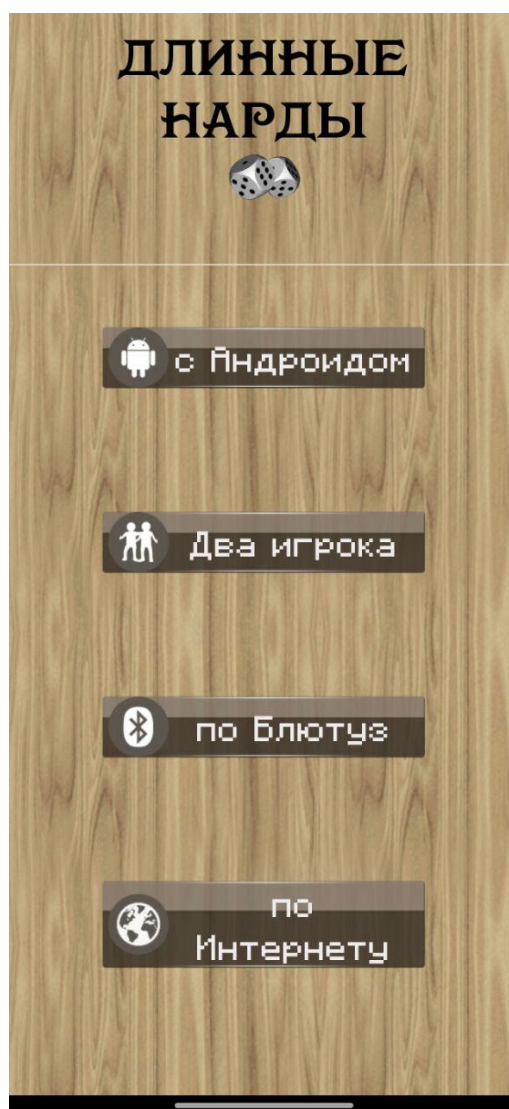


Рисунок 15 – интерфейс меню



Рисунок 16 – интерфейс игрового поля

– Выбор соперника (рисунок 15)

- При бросании кубиков подсвечиваются места куда можно поставить фишку (рисунок 16)
- Можно играть без подключения к интернету

Минусы:

- Реклама (рисунок 16)

4.2 Разработка структуры приложения и алгоритмов функционирования

После игрового меню генерируется игровое поле. В центре окна генерируется прямоугольник, на котором чередуются светлые и темные треугольники, в центре игральные кости. По боками расположены шашки чёрного и белого цвета, а над игровым полем – таймер (Рисунок 1).

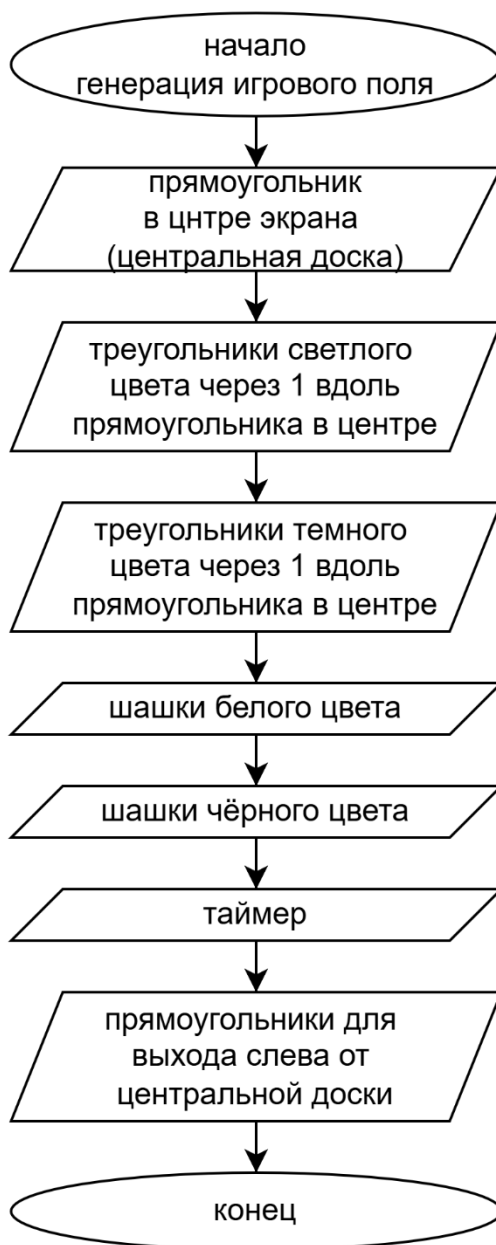


Рисунок 1 – генерация поля

Для начала игры создается игровое поле (Рисунок 1), далее генерируются кубики, на которых случайно появляются условные обозначения в виде точек в количестве от 1 до 6. Перед началом хода проверяется, может ли шашка переместиться на количество пунктов, выпавшее на двух кубиках или на одном кубике. Если ход возможен – шашка ходит; если нет – право хода передается другой шашке. Для определения

победителя подсчитывается, шашек какого цвета осталось больше – белого или черного (Рисунок 2).

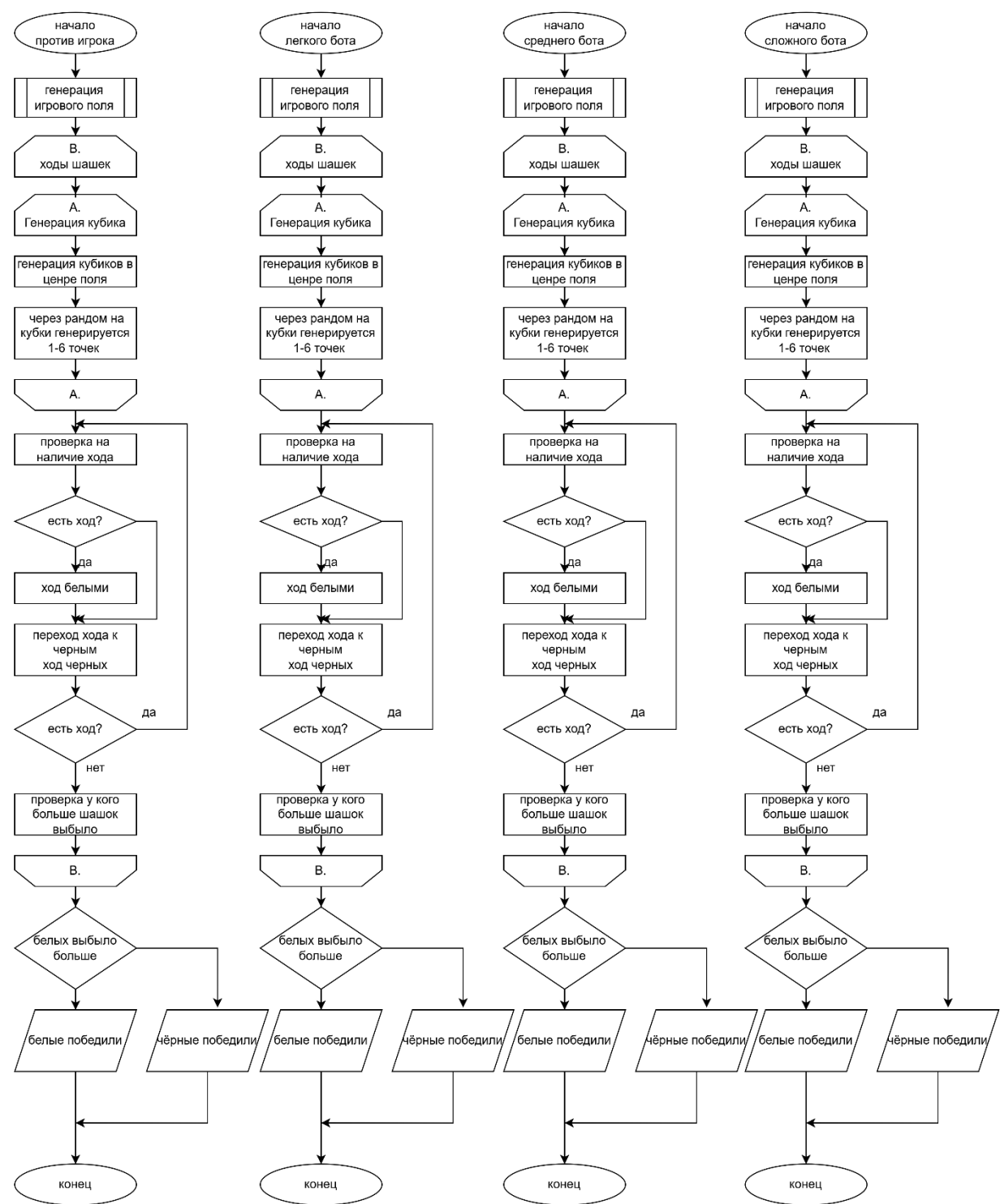
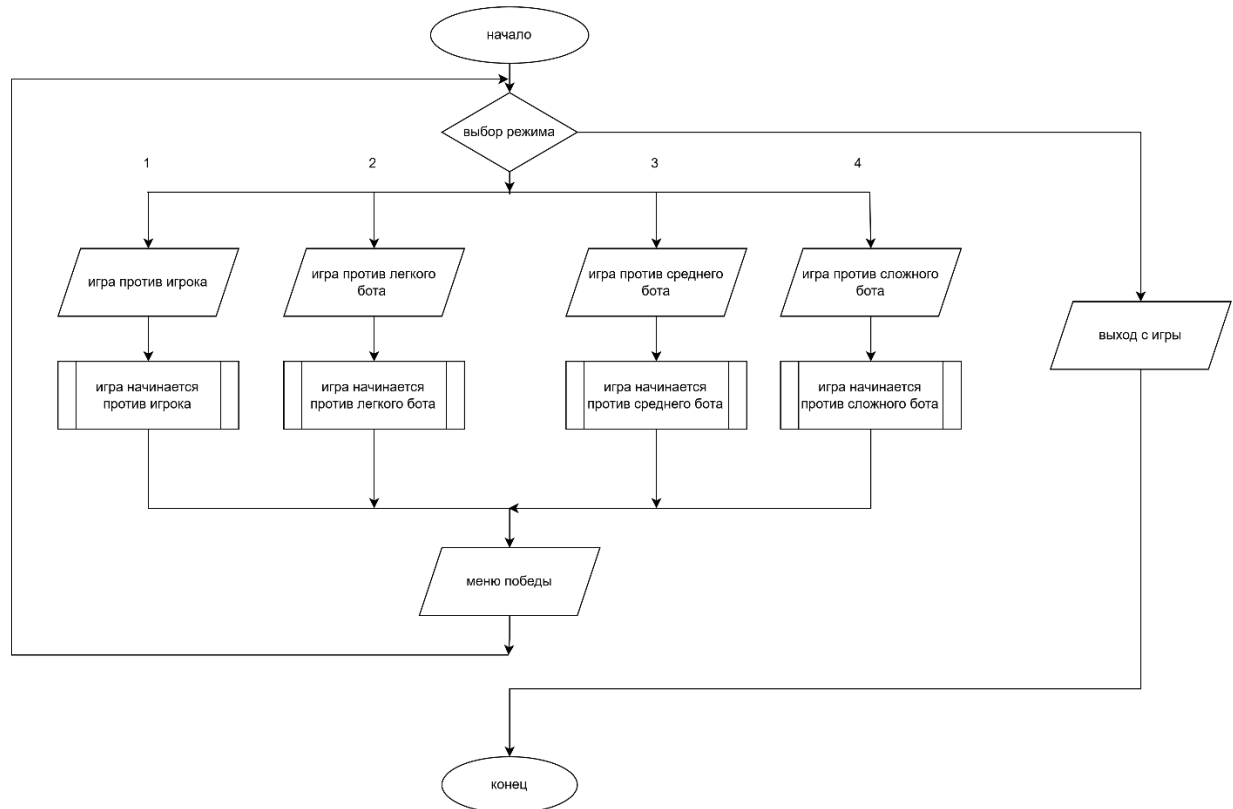


Рисунок 2 – логика игры

Есть 2 режима игры

- 1) Против человека – на 1 устройстве играет 2 человека
- 2) Против бота – игра против бота разного уровня сложности (легкий-средний-сложный)

Для выхода из игры есть кнопка «Выход с игры»



4.3 Разработка алгоритмов работы программы и программного обеспечения

В данном разделе будут произведены разработки алгоритмов функционирования игры «Длинные нарды», разработан интерфейс программы и реализованы её основные вычислительные и графические функции.

4.3.1 Разработка программного обеспечения

Программное обеспечение разработано на языке Python с использованием графической библиотеки Tkinter. Логика приложения инкапсулирована в главном классе LongNardMaster.

Точкой входа в приложение является инициализация главного окна Tk, после чего создаётся экземпляр управляющего класса, и запускается главный

цикл обработки событий `mainloop`, отвечающий за обработку кликов, отрисовку и анимации. Без данного блока программа не сможет запуститься как самостоятельное приложение.

Функция `__init__` является конструктором класса и вызывается автоматически при создании экземпляра программы, обеспечивая первичную инициализацию всех основных компонентов. В ходе её выполнения происходит привязка главного окна к объекту класса, задание заголовка окна, а также активация полноэкранного режима, что позволяет улучшить пользовательский опыт за счёт максимального использования экранного пространства. Дополнительно функция вызывает метод `update_screen_dims` для расчёта размеров интерфейса. Исходный код функции `__init__` представлен на рисунке 1.

```
def __init__(self, root):  
  
    self.root = root  
  
    self.root.title("Длинные Нарды - Премиум издание")  
  
    self.is_fullscreen = True  
  
    self.root.attributes("-fullscreen", self.is_fullscreen)  
  
    self.update_screen_dims()
```

Рисунок 1 – Исходный код функции `__init__`

Функция `reset_game_state` отвечает за установку начального состояния игры перед началом новой партии. В ходе её выполнения создаётся структура данных игрового поля из 24 позиций, каждая из которых хранит информацию

о количестве шашек и их принадлежности игроку. При размещении шашек в стартовые позиции строго соблюдается расстановка: белые находятся в левом нижнем углу доски, а черные располагаются в правом верхнем. Также обнуляются счётчики выброшенных шашек и сбрасываются вспомогательные параметры состояния анимации. Исходный код представлен на рисунке 2.

```
def reset_game_state(self):  
  
    self.board = [[0, None] for _ in range(24)]  
  
    self.board[0] = [15, 'w']  
  
    self.board[12] = [15, 'b']
```

Рисунок 2 – Исходный код функции reset_game_state

Функция calc_moves реализует основную игровую логику и отвечает за вычисление всех возможных ходов для выбранной шашки. Функция определяет цвет шашки, проверяет её принадлежность текущему игроку и вычисляет допустимые варианты перемещения с учётом значений выпавших кубиков. Важным алгоритмическим условием является то, что движение и белых, и черных шашек осуществляется строго против часовой стрелки. Дополнительно учитываются ограничения на движение с «головы», нахождение шашек в доме и возможность их выброса за пределы поля. Исходный код представлен на рисунке 3.

```
def calc_moves(self, start):  
  
    moves = {}  
  
    color = self.board[start][1]
```

Рисунок 4.3.3 – Исходный код функции calc_moves

Для реализации алгоритма искусственного интеллекта используется функция bot_move. Функция перебирает все позиции на игровом поле, определяет шашки текущего игрока и рассчитывает для них возможные ходы.

В зависимости от заданного уровня сложности алгоритм выбирает случайный ход (для лёгкого уровня), частично оптимальный (для среднего) или наилучший возможный (для сложного уровня). Исходный код представлен на рисунке 4.

```
def bot_move(self):  
  
    if not self.game_active:  
  
        return  
  
    # Выбор хода из all_moves
```

Рисунок 4 – Исходный код функции Bot_move

4.3.4 Описание экранных форм

Пользовательский интерфейс приложения спроектирован с использованием библиотеки Tkinter и ориентирован на интуитивно понятное взаимодействие. Ниже приведены описания основных графических окон программы.

4.3.4.1 Главное меню игры.

Главное меню является стартовым экраном приложения. Здесь расположены основные элементы навигации, позволяющие выбрать режим игры или выйти из приложения. Оформление выполнено в теплых тонах с использованием древесных текстур для создания атмосферы классической настольной игры. Главное меню представлено на рисунке 5. Все кнопки реализованы с помощью кнопки действий «Button», а надпись с помощью «Text».

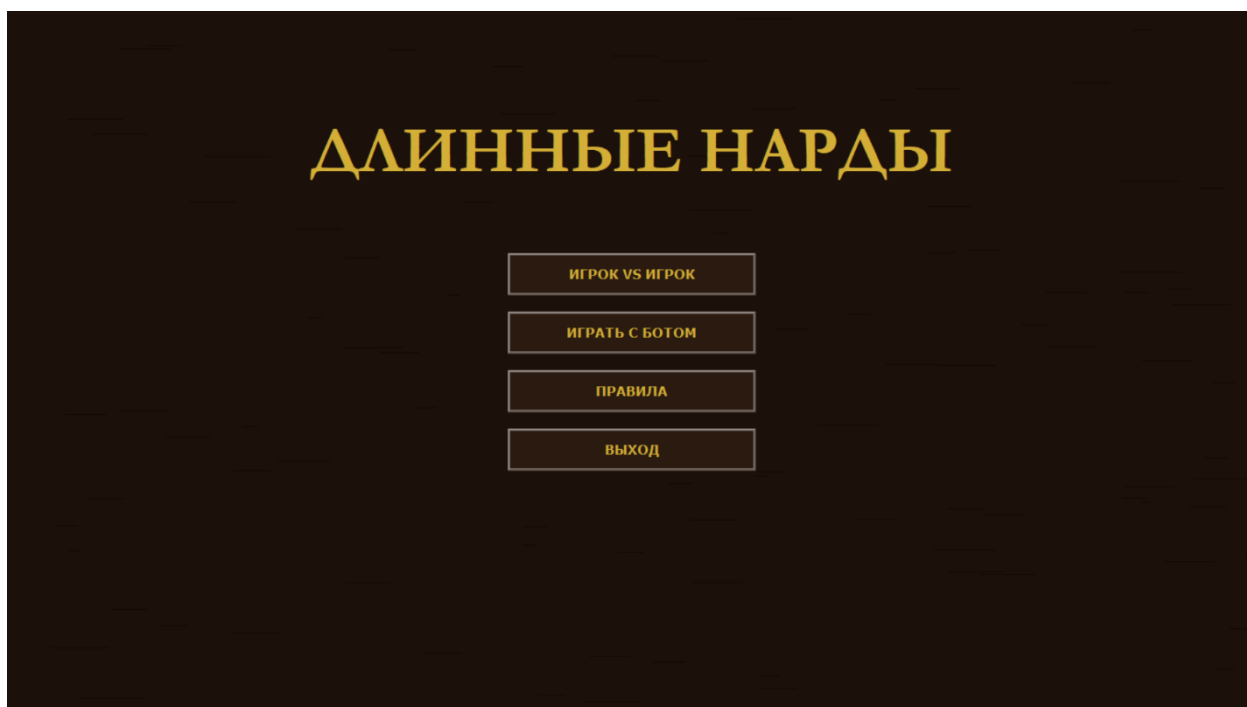


Рисунок 5 – Главное меню

4.3.4.2 Окно выбора режима игры против бота.

При выборе режима «Играть с ботом» открывается диалоговое окно настройки сложности. Пользователю предлагается три уровня: «Легкий», «Средний» и «Сложный» которые реализованы с помощью «OptionMenu», значения которых считываются программным методом «get()». Это позволяет адаптировать игровой процесс под уровень подготовки игрока. Окно настройка сложности окна представлено на рисунке 6. Чтобы начать игру необходимо нажать на кнопку «Начать», событийно привязанная через «bind»

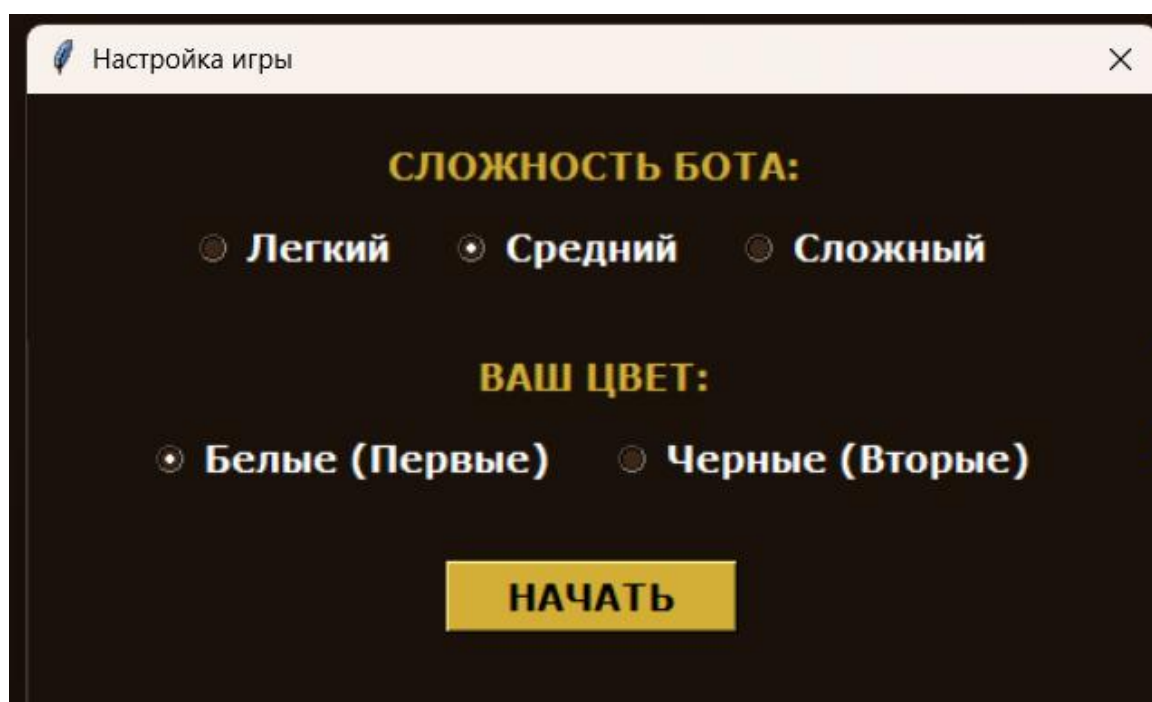


Рисунок 6 – Настройка сложности бота

4.3.4.3 Игровое поле.

Игровое поле, реализованное с помощью модуля «Canvas» – основная рабочая область программы. Включает в себя 24 лунки, отрисованные в виде вытянутых треугольников. В левой части экрана отображаются информационные панели (текущий ход, значения кубиков), а центральная часть отведена под анимацию перемещения шашек. Интерфейс игрового поля представлен на рисунке 7. Информационные метки, показывающие игроку чей сейчас ход, против кого он играет, сколько шашек выброшено и счёт,

реализованы с помощью «Label». Кнопка сдать с помощью «Button». Все элементы игрового поля управляются с помощью корневого окна «Tk».

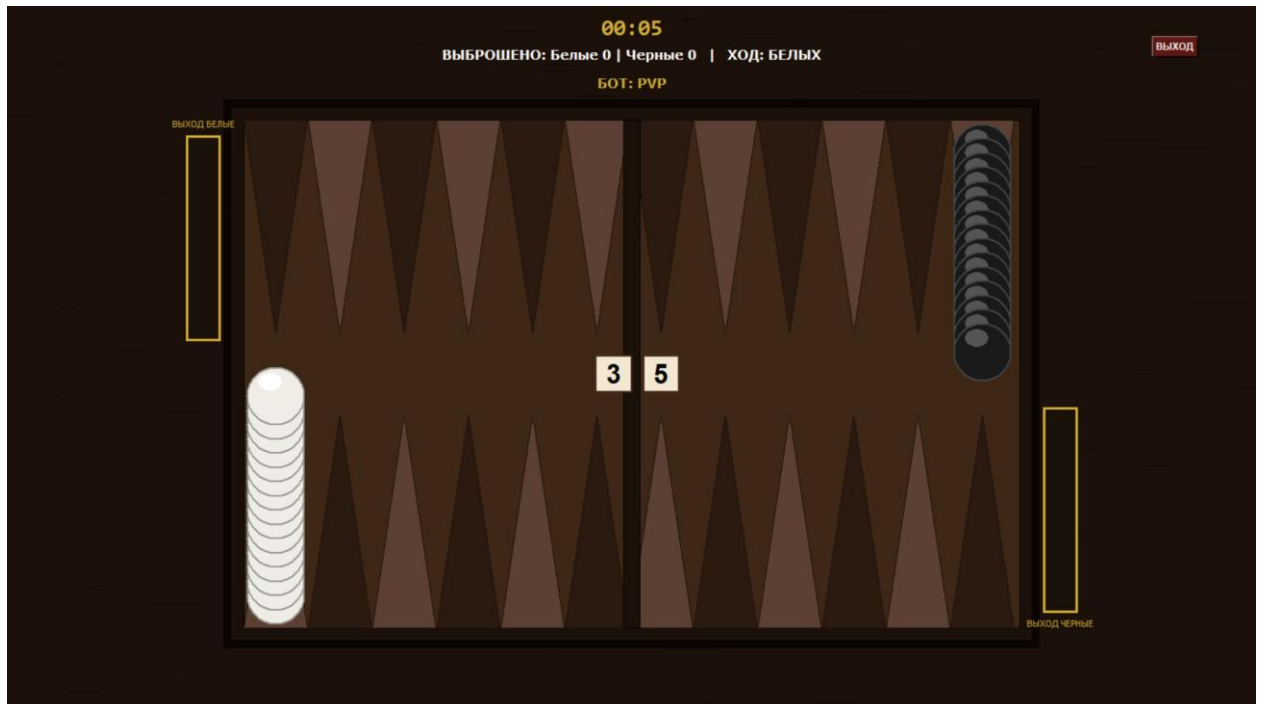


Рисунок 7 – Основной интерфейс игрового процесса

4.3.4.4 Меню подтверждения завершения игры (кнопка «Сдаться»).

В процессе игры пользователю доступна функция досрочного завершения партии. При нажатии кнопки «Сдаться» реализованной с помощью «Button», появляется модальное окно, уточняющее намерения игрока, что предотвращает случайное прерывание матча. Кнопки «сдаться» и «отмена» тоже реализованный с помощью «Button». Меню подтверждения выхода/сдачи представлено на рисунке 8.

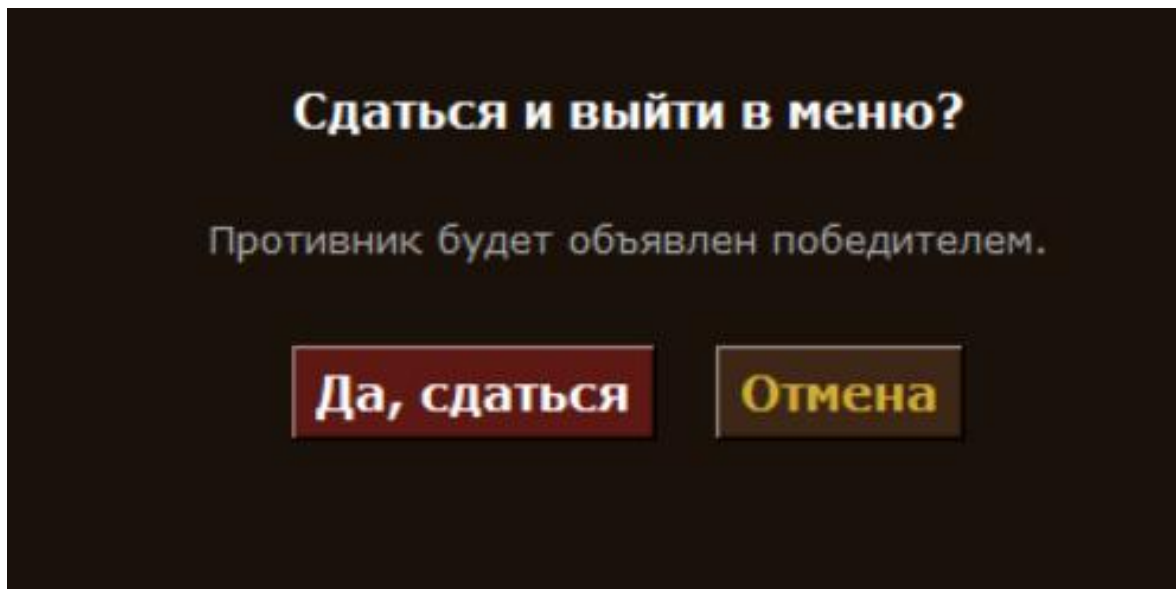


Рисунок 8 – Окно подтверждения выхода/сдачи

4.3.4.5 Окно результатов (победителя).

Окно результатов выводится автоматически при выполнении условий победы (когда один из игроков вывел все шашки за борт или сдался). Окно информирует о том, какой цвет шашек, и предлагает вернуться в главное меню для начала новой сессии. Все элементы текста реализованны с помощью «Text», а кнопка «В главное меню» с помощью «Button». Окно результатов представлено на рисунке 9.

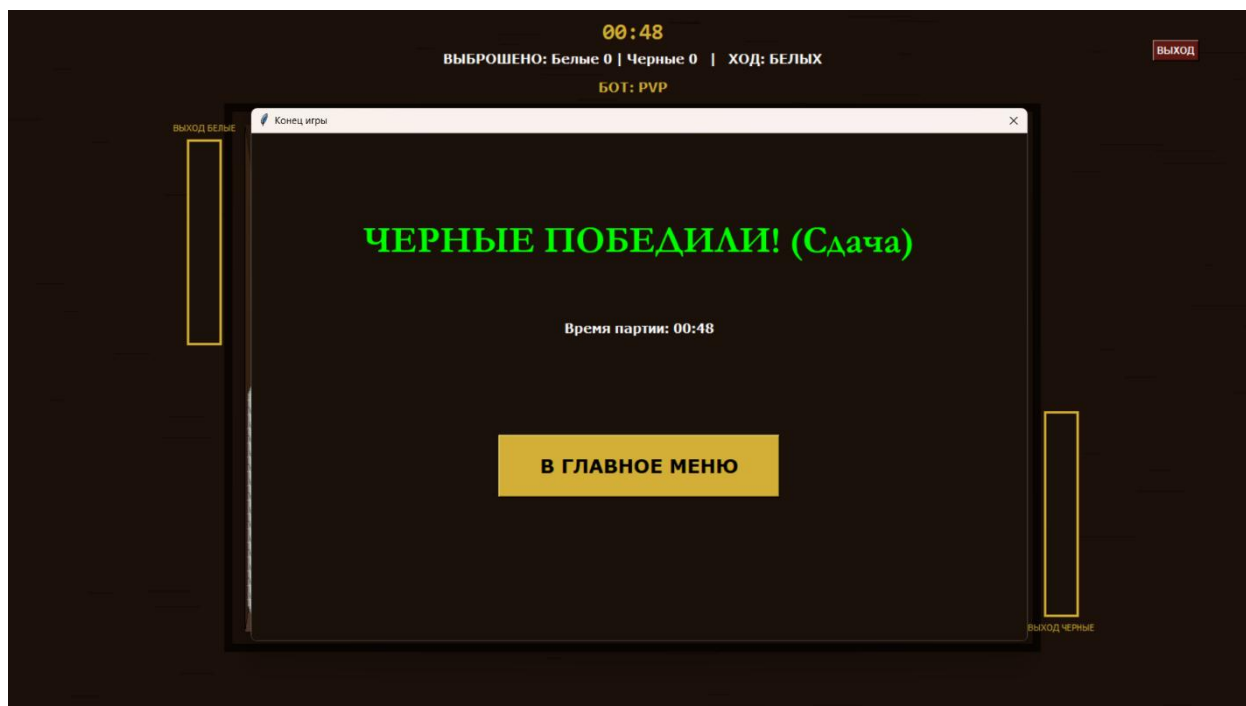


Рисунок 9 – Информационное окно завершения партии

4.3.4.6 Правила игры

Окно правил игры (рис. 10) представляет собой модальное диалоговое окно, реализованное с помощью «Toplevel» с темно-коричневым фоном и золотым шрифтом, полностью соответствующим общей стилистике приложения. Для заголовка "правила длинных нард", который вынесен за пределы основного текстового блока и текста самих правил используется виджет «Text». Информация внутри окна структурирована по логическим разделам, каждый из которых обозначен соответствующей графической иконкой, что облегчает восприятие.



Рисунок 10 – Окно правил игры

С правой стороны окно снабжено вертикальной полосой прокрутки «scrollbar» через связку параметров «yscrollcommand» у текстового поля и метода «set» у полосы прокрутки, что обеспечивает навигацию по всему объему текста правил. В нижней части окна расположена интерактивная кнопка «Button» «заккрыть», выполненная в стиле кнопок главного меню (Рис. 5), для возврата к предыдущему экрану. Данное окно играет ключевую роль в повышении доступности программы для новых пользователей, снижая порог вхождения.

ПРИЛОЖЕНИЕ А

(исходный код)

```
from __future__ import annotations

import copy
import math
import random
import time
import tkinter as tk

from dataclasses import dataclass, field
from typing import Any, Dict, List, Optional, Tuple, Union

# --- Константы геометрии и игры -----

POINTS_COUNT = 24
CHECKERS_PER_PLAYER = 15
WHITE = "w"
BLACK = "b"

# Порядок обхода: белые 0→23, чёрные 12→23 затем 0→11
PATH_WHITE: Tuple[int, ...] = tuple(range(POINTS_COUNT))
PATH_BLACK: Tuple[int, ...] = tuple(range(12, POINTS_COUNT)) + tuple(range(12))

# Индекс в пути: с этого шага и дальше — «дом» (последние 6 пунктов до выброса)
HOME_START_PATH_INDEX = 18

# Анимация хода шашки (линейная интерполяция, 30–60 кадров)
ANIM_MOVE_FRAMES = 60
ANIM_FPS_MS = 1000 // 60

# Теги слоёв canvas
TAG_SHELL = "shell"
```

TAG_PIECES = "pieces"

TAG_HINTS = "hints"

TAG_FLOATING = "floating"

TAG_CENTER_BAR = "center_bar"

--- Состояние партии -----

@dataclass

class GameState:

"""Чистые данные партии; без Tkinter и без методов отрисовки."""

board: List[List[Any]] # [count, color[None]] на каждой точке

dice: List[int]

current_turn: str

first_turn: Dict[str, bool] = field(default_factory=lambda: {WHITE: True, BLACK: True})

head_taken_this_turn: int = 0

out_count: Dict[str, int] = field(default_factory=lambda: {WHITE: 0, BLACK: 0})

is_double: bool = False

def copy(*self*) -> "GameState":

return GameState(

board=copy.deepcopy(*self*.board),

dice=*self*.dice[:],

current_turn=*self*.current_turn,

first_turn=dict(*self*.first_turn),

head_taken_this_turn=*self*.head_taken_this_turn,

out_count=dict(*self*.out_count),

is_double=*self*.is_double,

)

def initial_board() -> List[List[Any]]:

b = [[0, None] for _ in range(POINTS_COUNT)]

b[0] = [CHECKERS_PER_PLAYER, WHITE]

```
b[12] = [CHECKERS_PER_PLAYER, BLACK]
return b
```

```
def new_game_state() -> GameState:
```

```
    return GameState(
        board=initial_board(),
        dice=[],
        current_turn=WHITE,
        first_turn={WHITE: True, BLACK: True},
        head_taken_this_turn=0,
        out_count={WHITE: 0, BLACK: 0},
        is_double=False,
    )
```

```
# --- Логика правил (без UI) -----
```

```
def _path_for(color: str) -> Tuple[int, ...]:
```

```
    return PATH_WHITE if color == WHITE else PATH_BLACK
```

```
def _all_in_home(state: GameState, color: str) -> bool:
```

```
    path = _path_for(color)
    for i in range(POINTS_COUNT):
        cnt, col = state.board[i]
        if col == color and path.index(i) < HOME_START_PATH_INDEX:
            return False
    return True
```

```
def _farthest_from_home_path_index(state: GameState, color: str) -> int:
```

```
    """Минимальный индекс на пути среди шашек в доме (для правила выброса «с
    дальней»).""""
```

```
    path = _path_for(color)
    best: Optional[int] = None
    for i in range(POINTS_COUNT):
        cnt, col = state.board[i]
```

```

    if col != color or cnt <= 0:
        continue
    idx = path.index(i)
    if idx < HOME_START_PATH_INDEX:
        continue
    if best is None or idx < best:
        best = idx
    return -1 if best is None else best

```

```

def _dice_subsets(dice: List[int]) -> List[Tuple[int, List[int]]]:

```

```

    """

```

Варианты хода одной шашкой: одна кость или сумма двух разных костей.

Дубль: 1..min(4, len(dice)) одинаковых значений.

Возвращает (сумма шагов, список значений костей для списания).

```

    """

```

```

    if not dice:

```

```

        return []

```

```

    n = len(dice)

```

```

    out: List[Tuple[int, List[int]]] = []

```

```

    seen: set = set()

```

```

def add(cost: int, used: List[int]) -> None:

```

```

    key = (cost, tuple(sorted(used)))

```

```

    if key not in seen:

```

```

        seen.add(key)

```

```

        out.append((cost, used))

```

```

if len(set(dice)) == 1:

```

```

    v = dice[0]

```

```

    for k in range(1, min(4, n) + 1):

```

```

        add(k * v, [v] * k)

```

```

    return out

```

```

for i in range(n):
    add(dice[i], [dice[i]])
for i in range(n):
    for j in range(i + 1, n):
        if dice[i] != dice[j]:
            add(dice[i] + dice[j], [dice[i], dice[j]])
return out

```

```

def _can_take_from_head(state: GameState, color: str) -> bool:

```

```

    """С головы за ход — не более одной шашки, кроме первого хода при дубле (две)."""
    if state.head_taken_this_turn == 0:
        return True
    if state.first_turn[color] and state.is_double and state.head_taken_this_turn < 2:
        return True
    return False

```

```

def legal_moves_from_point(state: GameState, start: int) -> Dict[Union[int, str], List[int]]:

```

```

    """
    Все геометрически допустимые ходы с точки start с учётом головы и дома.
    Ключ: индекс точки или 'out'; значение: список значений костей (как в state.dice).
    """

```

```

    moves: Dict[Union[int, str], List[int]] = {}

```

```

    cnt, color = state.board[start]

```

```

    if cnt <= 0 or color != state.current_turn:

```

```

        return moves

```

```

    path = _path_for(color)

```

```

    start_idx = path.index(start)

```

```

    if start_idx == 0 and not _can_take_from_head(state, color):

```

```

        return moves

```

```

    all_home = _all_in_home(state, color)

```

```
farthest_idx = _farthest_from_home_path_index(state, color) if all_home else -1
```

```
for cost, dice_used in _dice_subsets(state.dice):
```

```
    target_idx = start_idx + cost
```

```
    if target_idx >= POINTS_COUNT:
```

```
        if not all_home:
```

```
            continue
```

```
        dist_to_end = POINTS_COUNT - start_idx
```

```
        if cost == dist_to_end:
```

```
            key: Union[int, str] = "out"
```

```
            if key not in moves or len(dice_used) < len(moves[key]):
```

```
                moves[key] = dice_used
```

```
        elif cost > dist_to_end and start_idx == farthest_idx:
```

```
            key = "out"
```

```
            if key not in moves or len(dice_used) < len(moves[key]):
```

```
                moves[key] = dice_used
```

```
    else:
```

```
        target_physical = path[target_idx]
```

```
        t_cnt, t_col = state.board[target_physical]
```

```
        if t_col in (None, color):
```

```
            if target_physical not in moves or len(dice_used) < len(moves[target_physical]):
```

```
                moves[target_physical] = dice_used
```

```
    return moves
```

```
def remove_dice_values(dice: List[int], used: List[int]) -> List[int]:
```

```
    """Снимает по одному вхождению каждого значения из used (мультимножество)."""
```

```
    pool = dice[:]
```

```
    for v in used:
```

```
        if v in pool:
```

```
            pool.remove(v)
```

```
    return pool
```

```
def apply_move(state: GameState, start: int, end: Union[int, str], dice_used: List[int]) -> GameState:
```



```
"""Возвращает новое состояние после хода (копия)."""
```

```
s = state.copy()
```

```
path = _path_for(s.current_turn)
```

```
if path.index(start) == 0:
```

```
    s.head_taken_this_turn += 1
```

```
s.board[start][0] -= 1
```

```
if s.board[start][0] == 0:
```

```
    s.board[start][1] = None
```

```
if end == "out":
```

```
    s.out_count[s.current_turn] += 1
```

```
else:
```

```
    s.board[end][1] = s.current_turn
```

```
    s.board[end][0] += 1
```

```
s.dice = remove_dice_values(s.dice, dice_used)
```

```
return s
```

```
def max_dice_usable(state: GameState) -> int:
```

```
    """
```

```
    Максимальное число «костей» (элементов списка dice), которые можно сыграть  
    за оставшуюся часть хода (рекурсия по всем легальным продолжениям).
```

```
    """
```

```
if not state.dice:
```

```
    return 0
```

```
color = state.current_turn
```

```
best = 0
```

```
for start in range(POINTS_COUNT):
```

```
    if state.board[start][1] != color:
```

```
        continue
```

```
    for target, used in legal_moves_from_point(state, start).items():
```

```
        gain = len(used)
```

```

    nxt = apply_move(state, start, target, used)
    total = gain + max_dice_usable(nxt)
    if total > best:
        best = total
return best

```

```
def legal_moves_maximal_first(
```

```
    state: GameState, start: int
```

```
) -> Dict[Union[int, str], List[int]]:
```

```
    """Только те ходы с start, которые входят в хотя бы одно максимальное по костям
    продолжение."""
```

```
    baseline = max_dice_usable(state)
```

```
    if baseline == 0:
```

```
        return {}
```

```
    out: Dict[Union[int, str], List[int]] = {}
```

```
    for target, used in legal_moves_from_point(state, start).items():
```

```
        gain = len(used)
```

```
        nxt = apply_move(state, start, target, used)
```

```
        if gain + max_dice_usable(nxt) == baseline:
```

```
            out[target] = used
```

```
    return out
```

```
def player_has_any_legal_move(state: GameState) -> bool:
```

```
    color = state.current_turn
```

```
    for i in range(POINTS_COUNT):
```

```
        if state.board[i][1] == color and legal_moves_maximal_first(state, i):
```

```
            return True
```

```
    return False
```

```
# --- Бот (без Tkinter): вход — GameState, выход — (start, target, dice_used) --
```

```
def _evaluate_position(state: GameState, for_color: str) -> float:
```

```
    """
```

```
    Простая эвристика: выброшенные шашки, прогресс по пути, концентрация в доме,
```

штраф за шашки соперника ближе к выбросу.

"""

path = _path_for(for_color)

opp = BLACK if for_color == WHITE else WHITE

opp_path = _path_for(opp)

score = state.out_count[for_color] * 200.0

for i in range(POINTS_COUNT):

 cnt, col = state.board[i]

 if cnt <= 0:

 continue

 idx = path.index(i) if col == for_color else None

 if col == for_color:

 score += idx * 3.0

 if idx >= HOME_START_PATH_INDEX:

 score += 15.0

 elif col == opp:

 oi = opp_path.index(i)

 score -= oi * 2.0

 if oi >= HOME_START_PATH_INDEX:

 score -= 25.0

return score

def enumerate_all_first_moves(state: GameState) -> List[Tuple[int, Union[int, str], List[int]]]:

 moves: List[Tuple[int, Union[int, str], List[int]]] = []

 for start in range(POINTS_COUNT):

 if state.board[start][1] != state.current_turn:

 continue

 for target, used in legal_moves_maximal_first(state, start).items():

 moves.append((start, target, used))

return moves

```
def choose_bot_move(state: GameState, level: str) -> Optional[Tuple[int, Union[int, str],
List[int]]]:
```

```
    """
```

```
    Выбор хода бота. Не трогает UI.
```

```
    level: 'easy' | 'medium' | 'hard'
```

```
    """
```

```
    all_moves = enumerate_all_first_moves(state)
```

```
    if not all_moves:
```

```
        return None
```

```
def score_after_one_step(m: Tuple[int, Union[int, str], List[int]]) -> float:
```

```
    start, target, used = m
```

```
    nxt = apply_move(state, start, target, used)
```

```
    return _evaluate_position(nxt, state.current_turn)
```

```
if level == "easy":
```

```
    return random.choice(all_moves)
```

```
if level == "medium":
```

```
    if random.random() < 0.35:
```

```
        return random.choice(all_moves)
```

```
    return max(all_moves, key=score_after_one_step)
```

```
# hard
```

```
return max(all_moves, key=score_after_one_step)
```

```
# --- UI приложение -----
```

```
class LongNardMaster:
```

```
    def __init__(self, root):
```

```
        self.root = root
```

```
        self.root.title("Длинные Нарды - Премиум издание")
```

```
        self.is_fullscreen = True
```

```
        self.root.attributes("-fullscreen", self.is_fullscreen)
```

```
self.update_screen_dims()
```

```
self.clr_wood = "#1c110a"
```

```
self.clr_board = "#3e2716"
```

```
self.clr_accent = "#d4af37"
```

```
self.clr_white = "#f0ede6"
```

```
self.clr_black = "#1a1a1a"
```

```
self.clr_selected = "#00cc66"
```

```
self.canvas = tk.Canvas(
```

```
    root,
```

```
    width=self.sw,
```

```
    height=self.sh,
```

```
    bg=self.clr_wood,
```

```
    highlightthickness=0,
```

```
)
```

```
self.canvas.pack(fill="both", expand=True)
```

```
self.timer_id = None
```

```
self.game_active = False
```

```
self.state: Optional[GameState] = None
```

```
self._shell_drawn = False
```

```
self.show_menu()
```

```
def update_screen_dims(self):
```

```
    self.sw = self.root.winfo_screenwidth()
```

```
    self.sh = self.root.winfo_screenheight()
```

```
    base = min(self.sw, self.sh) * 0.85
```

```
    self.board_h = base * 0.85
```

```
    # Возвращаем исходную геометрию поля
```

```
self.board_w = base * 1.3
```

```
self.margin_x = (self.sw - self.board_w) / 2
```

```
self.margin_y = (self.sh - self.board_h) / 2 + 20
```

```
self.point_w = int(self.board_w // 12)
```

```
self.piece_r = max(8, self.point_w // 2 - 4)
```

```
# Отступ зон "выхода" от края доски (чуть дальше наружу)
```

```
self.out_pad = max(60, int(self.point_w * 0.9))
```

```
# Центральная перегородка: базовая ширина + асимметрия (справа тоньше на 3px)
```

```
self.center_bar_w = min(14, max(10, int(self.point_w * 0.35) - 3))
```

```
self.center_bar_right_trim = 5 # справа ещё на 2px тоньше
```

```
half = self.center_bar_w / 2
```

```
# справа уменьшаем половинку, но не даём стать слишком тонкой
```

```
self.center_bar_left_half = half
```

```
self.center_bar_right_half = max(4.0, half - float(self.center_bar_right_trim))
```

```
self.font_ui = ("Verdana", max(10, int(self.sh * 0.015)), "bold")
```

```
self.font_title = ("Garamond", max(24, int(self.sh * 0.07)), "bold")
```

```
def toggle_fullscreen(self, event=None):
```

```
    self.is_fullscreen = not self.is_fullscreen
```

```
    self.root.attributes("-fullscreen", self.is_fullscreen)
```

```
    self.root.after(100, self.rebuild_ui)
```

```
def rebuild_ui(self):
```

```
    self.update_screen_dims()
```

```
    self._shell_drawn = False
```

```
    if self.game_active and self.state is not None:
```

```
        self.draw_board(full_redraw=True)
```

```
        self.update_score_display()
```

```
    else:
```

```
        self.show_menu()
```

```

def reset_game_state(self):
    self.state = new_game_state()
    self.selected_point: Optional[int] = None
    self.valid_moves: Dict = {}
    self.is_animating = False
    self.start_game_time = time.time()
    self.waiting_for_dice = False
    self.game_active = True
    self._shell_drawn = False
    self._anim_pending: Optional[Tuple[int, Union[int, str], List[int]]] = None

```

```

def get_coords(self, index):
    if 0 <= index <= 11:
        x = self.margin_x + index * self.point_w + self.point_w // 2
        y = self.margin_y + self.board_h
        return x, y, -1
    col = 23 - index
    x = self.margin_x + col * self.point_w + self.point_w // 2
    y = self.margin_y
    return x, y, 1

```

```

def draw_bg_texture(self):
    self.canvas.delete("bg_tex")
    for _ in range(100):
        x1 = random.randint(0, self.sw)
        y1 = random.randint(0, self.sh)
        self.canvas.create_line(
            x1,
            y1,
            x1 + random.randint(10, 80),
            y1,
            fill="#120a05",
            width=1,

```

```
        tags="bg_tex",  
    )
```

```
def show_menu(self):
```

```
    self.game_active = False
```

```
    self.state = None
```

```
    if self.timer_id:
```

```
        self.root.after_cancel(self.timer_id)
```

```
    self.canvas.delete("all")
```

```
    self.draw_bg_texture()
```

```
    self.canvas.create_text(  
        self.sw // 2,
```

```
        self.sh * 0.2,
```

```
        text="ДЛИННЫЕ НАРДЫ",
```

```
        font=self.font_title,
```

```
        fill=self.clr_accent,
```

```
    )
```

```
btn_frame = tk.Frame(self.canvas, bg=self.clr_wood)
```

```
self.canvas.create_window(self.sw // 2, self.sh * 0.5, window=btn_frame)
```

```
btn_style = {
```

```
    "font": self.font_ui,
```

```
    "width": 25,
```

```
    "pady": 10,
```

```
    "padx": 10,
```

```
    "bg": "#2a1a10",
```

```
    "fg": self.clr_accent,
```

```
    "activebackground": "#4a301e",
```

```
    "activeforeground": "#fff",
```

```
    "relief": "ridge",
```

```
    "bd": 3,
```



```

        "cursor": "hand2",
    }

    tk.Button(btn_frame, text="ИГРОК VS ИГРОК", command=lambda:
self.start_game_init(None, WHITE), **btn_style).pack(pady=10)

    tk.Button(btn_frame, text="ИГРАТЬ С БОТОМ", command=self.show_bot_setup,
**btn_style).pack(pady=10)

    tk.Button(btn_frame, text="ПРАВИЛА", command=self.show_rules,
**btn_style).pack(pady=10)

    tk.Button(btn_frame, text="ВЫХОД", command=self.root.destroy,
**btn_style).pack(pady=10)

def show_bot_setup(self):
    win = tk.Toplevel(self.root)
    win.title("Настройка игры")
    win.geometry("500x350")
    win.configure(bg="#1a110a")
    win.transient(self.root)
    win.grab_set()

    self.bot_lvl_var = tk.StringVar(value="medium")
    self.color_var = tk.StringVar(value=WHITE)

    tk.Label(win, text="СЛОЖНОСТЬ БОТА:", font=self.font_ui, bg="#1a110a",
fg=self.clr_accent).pack(pady=(20, 10))

    f1 = tk.Frame(win, bg="#1a110a")
    f1.pack()

    for t, v in [("Легкий", "easy"), ("Средний", "medium"), ("Сложный", "hard")]:
        tk.Radiobutton(
            f1, text=t, variable=self.bot_lvl_var, value=v, font=self.font_ui, bg="#1a110a",
fg="white", selectcolor="#3e2716"
        ).pack(side="left", padx=10)

    tk.Label(win, text="БАШ ЦВЕТ:", font=self.font_ui, bg="#1a110a",
fg=self.clr_accent).pack(pady=(30, 10))

    f2 = tk.Frame(win, bg="#1a110a")

```

```

f2.pack()

tk.Radiobutton(
    f2, text="Белые (Первые)", variable=self.color_var, value=WHITE, font=self.font_ui,
    bg="#1a110a", fg="white", selectcolor="#3e2716"
).pack(side="left", padx=10)

tk.Radiobutton(
    f2, text="Черные (Вторые)", variable=self.color_var, value=BLACK, font=self.font_ui,
    bg="#1a110a", fg="white", selectcolor="#3e2716"
).pack(side="left", padx=10)

tk.Button(win, text="НАЧАТЬ", font=self.font_ui, bg="#d4af37", fg="black", padx=20,
    command=lambda: self.start_bot_game(win)).pack(pady=30)

def start_bot_game(self, window):
    self.bot_level = self.bot_lvl_var.get()
    self.human_color = self.color_var.get()
    window.destroy()
    self.start_game_init(self.bot_level, self.human_color)

def show_rules(self):
    win = tk.Toplevel(self.root)
    win.title("Правила Игры")
    win.geometry("900x750")
    win.minsize(500, 400)
    win.configure(bg="#1a110a")
    win.transient(self.root)
    win.grab_set()

    title = tk.Label(win, text="ПРАВИЛА ДЛИННЫХ НАРД", font=("Garamond", 20, "bold"),
        bg="#1a110a", fg=self.clr_accent)
    title.pack(pady=10)

    text_frame = tk.Frame(win, bg="#1a110a")
    text_frame.pack(fill="both", expand=True, padx=20, pady=10)

```

```
text_widget = tk.Text(text_frame, wrap="word", font=("Verdana", 11), bg="#2a1a10",  
fg="white", padx=15, pady=15, relief="flat")
```

```
scrollbar = tk.Scrollbar(text_frame, command=text_widget.yview)
```

```
text_widget.configure(yscrollcommand=scrollbar.set)
```

```
scrollbar.pack(side="right", fill="y")
```

```
text_widget.pack(side="left", fill="both", expand=True)
```

```
rules_text = (
```

```
" 📖 ПРАВИЛА ИГРЫ «ДЛИННЫЕ НАРДЫ»\n\n"
```

```
" 🎯 ЦЕЛЬ ИГРЫ:\n"
```

```
"Вывести все 15 своих шашек с доски раньше соперника.\n\n"
```

```
" 🎲 НАЧАЛЬНАЯ РАССТАНОВКА:\n"
```

```
"• У каждого игрока по 15 шашек.\n"
```

```
"• Белые начинают игру и располагаются в лунке 0.\n"
```

```
"• Черные располагаются в лунке 12.\n"
```

```
"• Все шашки каждого игрока стоят в одной стартовой позиции («голове»).\n\n"
```

```
" 🎲 ХОД И КУБИКИ:\n"
```

```
"• Игроки ходят по очереди.\n"
```

```
"• В начале хода бросаются 2 кубика.\n"
```

```
"• Числа на кубиках — это количество шагов, на которое можно передвинуть  
шашки.\n"
```

```
"• Можно использовать каждое число отдельно или суммарно.\n"
```

```
"• Если выпал дубль (например 3-3), игрок делает 4 хода этим числом.\n\n"
```

```
" ➡ ДВИЖЕНИЕ ШАШЕК:\n"
```

```
"• Все шашки движутся против часовой стрелки.\n"
```

```
"• Белые: от 0 → 23.\n"
```

```
"• Черные: от 12 → 23 → 0 → 11.\n"
```

```
"• Шашки можно ставить только на:\n"
```

```
" – пустую лунку\n"
```

```
" – или лунку со своими шашками\n"
```

```
"• Ставить на лунку, занятую соперником, запрещено.\n\n"
```

```
" 🚫 БЛОКИРОВКА:\n"
```

- Если все возможные ходы заблокированы, игрок пропускает ход.\n\n"

❎ ПРАВИЛО «ГОЛОВЫ»:\n"

- «Голова» — стартовая лунка игрока.\n"
- За один ход можно снять только 1 шашку с головы.\n"
- Исключение: в самый первый ход при дубле (3-3, 4-4, 6-6)\n"
- " разрешено снять 2 шашки.\n\n"

🏠 ДОМ И ВЫВОД ШАШЕК (BEAR-OFF):\n"

- Вывод возможен только когда все шашки игрока находятся в доме.\n"
- Дом белых: лунки 18–23.\n"
- Дом черных: лунки 6–11.\n"
- Шашка выводится при точном значении кубика.\n"
- Если число больше — можно вывести самую дальнюю шашку.\n\n"

⚙️ ОСОБЕННОСТИ ХОДА:\n"

- Нужно использовать максимально возможное количество ходов.\n"
- Если можно сделать оба хода — их нужно сделать.\n"
- Если возможен только один — выполняется он.\n\n"

🎮 РЕЖИМЫ ИГРЫ:\n"

- Игрок vs Игрок — игра вдвоем.\n"
- Игрок vs Бот:\n"
- " — Легкий: случайные допустимые ходы\n"
- " — Средний: в основе эвристика, иногда ошибка\n"
- " — Сложный: лучший по эвристике первый шаг\n\n"

🏆 ПОБЕДА:\n"

- Побеждает игрок, который первым вывел все 15 шашек.\n"
- При выходе из игры текущий игрок считается проигравшим.\n\n"

💡 ПОДСКАЗКИ:\n"

- Выбирайте шашку кликом.\n"
- Доступные ходы подсвечиваются (используемые кости).\n"

)

text_widget.insert("1.0", rules_text)

text_widget.config(state="disabled")

```
tk.Button(win, text="ЗАКРЫТЬ", font=self.font_ui, bg="#3e2716", fg=self.clr_accent,
padx=20, command=win.destroy).pack(pady=15)
```

```
def start_game_init(self, bot_level, human_color):
```

```
    self.canvas.delete("all")
```

```
    self.draw_bg_texture()
```

```
    self.bot_level = bot_level
```

```
    self.human_color = human_color
```

```
    self.reset_game_state()
```

```
    self.canvas.bind("<Button-1>", self.on_click)
```

```
    self.timer_text_id = self.canvas.create_text(
```

```
        self.sw // 2,
```

```
        self.sh * 0.03,
```

```
        text="00:00",
```

```
        fill=self.clr_accent,
```

```
        font=("Consolas", int(self.sh * 0.025), "bold"),
```

```
        tags="ui",
```

```
    )
```

```
    self.score_text_id = self.canvas.create_text(self.sw // 2, self.sh * 0.07, text="", fill="white",
font=self.font_ui, tags="ui")
```

```
    mode_str = f"БОТ: {self.bot_level.upper() if self.bot_level else 'PVP'}"
```

```
    self.canvas.create_text(self.sw // 2, self.sh * 0.11, text=mode_str, fill=self.clr_accent,
font=self.font_ui, tags="ui")
```

```
    btn_quit = tk.Button(self.root, text="ВЫХОД", font=("Arial", 9, "bold"), bg="#5e1914",
fg="white", command=self.confirm_exit)
```

```
    self.canvas.create_window(self.sw - 100, 50, window=btn_quit, tags="ui")
```

```
    self.update_timer()
```

```
    self.draw_board(full_redraw=True)
```

```
    self.root.after(800, self.animate_dice)
```

```

def update_timer(self):
    if self.game_active:
        elapsed = int(time.time() - self.start_game_time)
        self.canvas.itemconfig(self.timer_text_id, text=f"{elapsed//60:02d}:{elapsed%60:02d}")
        self.timer_id = self.root.after(1000, self.update_timer)

def update_score_display(self):
    if hasattr(self, "score_text_id") and self.score_text_id and self.state:
        info = f"ВЫБРОШЕНО: Белые {self.state.out_count[WHITE]} | Черные {self.state.out_count[BLACK]}"
        info += f" | ХОД: {'БЕЛЫХ' if self.state.current_turn == WHITE else 'ЧЕРНЫХ'}"
        self.canvas.itemconfig(self.score_text_id, text=info)

def draw_piece(self, x, y, color, is_selected=False, tags="pieces"):
    main_col = self.clr_white if color == WHITE else self.clr_black
    highlight = self.clr_selected if is_selected else ("#999" if color == WHITE else "#444")
    r = self.piece_r

    self.canvas.create_oval(x - r, y - r, x + r, y + r, fill=main_col, outline=highlight, width=2,
tags=tags)
    if color == WHITE:
        self.canvas.create_oval(x - r * 0.6, y - r * 0.8, x + r * 0.2, y - r * 0.2, fill="#fff", outline="",
tags=tags)
    else:
        self.canvas.create_oval(x - r * 0.6, y - r * 0.8, x + r * 0.2, y - r * 0.2, fill="#555", outline="",
tags=tags)

def _draw_board_shell(self):
    """Рамка, треугольники, зона выброса, кубики — редко меняются."""
    self.canvas.delete(TAG_SHELL)
    self.canvas.create_rectangle(
        self.margin_x - 20,
        self.margin_y - 20,
        self.margin_x + self.board_w + 20,
        self.margin_y + self.board_h + 20,

```

```

        fill=self.clr_wood,
        outline="#0a0502",
        width=10,
        tags=TAG_SHELL,
    )
    self.canvas.create_rectangle(
        self.margin_x, self.margin_y, self.margin_x + self.board_w, self.margin_y + self.board_h,
        fill=self.clr_board, tags=TAG_SHELL
    )

```

```

for i in range(POINTS_COUNT):

```

```

    cx, cy, direction = self.get_coords(i)
    color = "#5c4033" if i % 2 == 0 else "#2a1a10"
    x_l, x_r = cx - self.point_w // 2, cx + self.point_w // 2
    h_tri = self.board_h * 0.42
    pts = [x_l, cy, x_r, cy, cx, cy - h_tri] if direction == -1 else [x_l, cy, x_r, cy, cx, cy + h_tri]
    self.canvas.create_polygon(pts, fill=color, outline="#1a110a", tags=TAG_SHELL)

```

```

mid_x = self.margin_x + self.board_w // 2
bar_l_half = getattr(self, "center_bar_left_half", self.center_bar_w / 2)
bar_r_half = getattr(self, "center_bar_right_half", self.center_bar_w / 2)
self.canvas.create_rectangle(
    mid_x - bar_l_half,
    self.margin_y,
    mid_x + bar_r_half,
    self.margin_y + self.board_h,
    fill=self.clr_wood,
    outline="black",
    tags=(TAG_SHELL, TAG_CENTER_BAR),
)

```

```

white_out_x, white_out_y = self.margin_x - self.out_pad, self.margin_y + 20
self.canvas.create_rectangle(
    white_out_x,

```

```

        white_out_y,
        white_out_x + 40,
        white_out_y + self.board_h * 0.4,
        fill="#1a110a",
        outline=self.clr_accent,
        width=3,
        tags=TAG_SHELL,
    )
    self.canvas.create_text(
        white_out_x + 20, white_out_y - 15, text="ВЫХОД БЕЛЫЕ", fill=self.clr_accent,
        font=("Arial", 8), tags=TAG_SHELL
    )

    black_out_x, black_out_y = (
        self.margin_x + self.board_w + (self.out_pad - 40),
        self.margin_y + self.board_h - 20 - self.board_h * 0.4,
    )
    self.canvas.create_rectangle(
        black_out_x,
        black_out_y,
        black_out_x + 40,
        black_out_y + self.board_h * 0.4,
        fill="#1a110a",
        outline=self.clr_accent,
        width=3,
        tags=TAG_SHELL,
    )
    self.canvas.create_text(
        black_out_x + 20,
        black_out_y + self.board_h * 0.4 + 15,
        text="ВЫХОД ЧЕРНЫЕ",
        fill=self.clr_accent,
        font=("Arial", 8),
        tags=TAG_SHELL,

```


)

```
def _draw_dice_on_shell(self, dice_vals: Optional[List[int]]):
    """Обновляет только прямоугольники кубиков на слое shell."""
    self.canvas.delete("dice_draw")

    if not dice_vals:
        return

    mid_x = self.margin_x + self.board_w // 2
    for idx, v in enumerate(dice_vals):
        d_size = self.sh * 0.05

        # Кубики по центру, но строго с зазором 5px от центральной перегородки.
        # Требование: расстояние от края перегородки до ближайшего края кубика = 5px.
        n = len(dice_vals)
        half_left = getattr(self, "center_bar_left_half", self.center_bar_w / 2)
        half_right = getattr(self, "center_bar_right_half", self.center_bar_w / 2)
        edge_gap = 5.0
        first_center_offset_left = half_left + edge_gap + (d_size / 2)
        first_center_offset_right = half_right + edge_gap + (d_size / 2)
        step = d_size + 12 # расстояние между кубиками на одной стороне

        if n == 1:
            centers = [mid_x] # теоретически не используется, но оставим красиво
        elif n == 2:
            centers = [mid_x - first_center_offset_left, mid_x + first_center_offset_right]
        elif n == 3:
            centers = [
                mid_x - (first_center_offset_left + step),
                mid_x - first_center_offset_left,
                mid_x + first_center_offset_right,
            ]
        else:
            # 4 и более: симметрично 2 слева + 2 справа (далее — в шахматном порядке
            # наружу)
            centers = [
```

```

        mid_x - (first_center_offset_left + step),
        mid_x - first_center_offset_left,
        mid_x + first_center_offset_right,
        mid_x + (first_center_offset_right + step),
    ]
    if n > 4:
        extra = n - 4
        for k in range(extra):
            dist = (first_center_offset_left if k % 2 == 0 else first_center_offset_right) + (2 + k
// 2) * step
            centers.append(mid_x - dist if k % 2 == 0 else mid_x + dist)

    x_center = centers[idx]
    dx = x_center - (d_size / 2)
    dy = self.margin_y + self.board_h / 2
    self.canvas.create_rectangle(
        dx,
        dy - d_size / 2,
        dx + d_size,
        dy + d_size / 2,
        fill="#f4e7d3",
        outline="#4a301e",
        width=2,
        tags=(TAG_SHELL, "dice_draw"),
    )
    self.canvas.create_text(
        dx + d_size / 2,
        dy,
        text=str(v),
        font=("Arial", int(d_size * 0.6), "bold"),
        fill="#000",
        tags=(TAG_SHELL, "dice_draw"),
    )

```

```

def draw_out_checkers(self, cx, start_y, count, color, direction, tags=TAG_PIECES):
    step = min(12, (self.board_h * 0.4 - 20) / max(15, 1))
    for i in range(count):
        y = start_y + (i * step) * direction
        main_col = self.clr_white if color == WHITE else self.clr_black
        self.canvas.create_oval(cx - 10, y - 10, cx + 10, y + 10, fill=main_col, outline="#555",
tags=tags)

```

```

def _triangle_height(self):
    return self.board_h * 0.42

```

```

def _stack_step(self, count: int) -> float:
    """Шаг укладки шашек как было раньше (сжатие по высоте треугольника)."""
    h_tri = self._triangle_height()
    return min(self.piece_r * 2.2, h_tri / max(count, 1))

```

```

def _stack_top_xy(self, point_index: int, count: int) -> Tuple[float, float]:
    """
    Экранные координаты верхней шашки в стопке.
    ВАЖНО: используем тот же шаг, что и в фактической отрисовке (`_stack_step`),
    иначе при больших стопках возникает «подвисание/рывок» при пересчёте.
    """
    cx, cy, direction = self.get_coords(point_index)
    if count <= 0:
        return float(cx), float(cy)
    step = self._stack_step(count)
    y = cy + (self.piece_r + 5 + (count - 1) * step) * direction
    return float(cx), float(y)

```

```

def _catmull_rom_polyline(self, pts: List[Tuple[float, float]], samples_per_seg: int = 18) ->
List[Tuple[float, float]]:

```

```

    """

```

Сглаживает путь через контрольные точки (Catmull–Rom) в полилинию.

Это убирает резкие «углы» на стыках и делает пересечение центра ровным.

```
"""
```

```
if not pts or len(pts) < 2:
```

```
    return pts[:]
```

```
if samples_per_seg < 4:
```

```
    samples_per_seg = 4
```

```
out: List[Tuple[float, float]] = []
```

```
n = len(pts)
```

```
def p(i: int) -> Tuple[float, float]:
```

```
    if i < 0:
```

```
        return pts[0]
```

```
    if i >= n:
```

```
        return pts[-1]
```

```
    return pts[i]
```

```
for i in range(n - 1):
```

```
    p0 = p(i - 1)
```

```
    p1 = p(i)
```

```
    p2 = p(i + 1)
```

```
    p3 = p(i + 2)
```

```
    for s in range(samples_per_seg):
```

```
        t = s / samples_per_seg
```

```
        t2 = t * t
```

```
        t3 = t2 * t
```

```
        # Catmull–Rom (tension = 0.5)
```

```
        x = 0.5 * (
```

```
            (2 * p1[0])
```

```
            + (-p0[0] + p2[0]) * t
```

```
            + (2 * p0[0] - 5 * p1[0] + 4 * p2[0] - p3[0]) * t2
```

```
            + (-p0[0] + 3 * p1[0] - 3 * p2[0] + p3[0]) * t3
```

```
        )
```

```
        y = 0.5 * (
```

```

        (2 * p1[1])
        + (-p0[1] + p2[1]) * t
        + (2 * p0[1] - 5 * p1[1] + 4 * p2[1] - p3[1]) * t2
        + (-p0[1] + 3 * p1[1] - 3 * p2[1] + p3[1]) * t3
    )
    if not out:
        out.append((x, y))
    else:
        # избегаем дублей соседних точек
        lx, ly = out[-1]
        if (x - lx) * (x - lx) + (y - ly) * (y - ly) > 1e-6:
            out.append((x, y))
    out.append(pts[-1])
    return out

```

```

def _quadratic_bezier_polyline(
    self,
    p0: Tuple[float, float],
    p1: Tuple[float, float],
    p2: Tuple[float, float],
    samples: int = 60,
) -> List[Tuple[float, float]]:
    """Квадратичная Bezier-кривая в полилинию (без углов/перелётов)."""
    if samples < 8:
        samples = 8
    out: List[Tuple[float, float]] = []
    for i in range(samples + 1):
        t = i / samples
        u = 1.0 - t
        x = (u * u) * p0[0] + 2 * u * t * p1[0] + (t * t) * p2[0]
        y = (u * u) * p0[1] + 2 * u * t * p1[1] + (t * t) * p2[1]
        if not out:
            out.append((x, y))

```

```

else:
    lx, ly = out[-1]
    if (x - lx) * (x - lx) + (y - ly) * (y - ly) > 1e-6:
        out.append((x, y))
return out

```

```

def _draw_pieces_layer(
    self,
    temp_dice: Optional[List[int]] = None,
    hide_source_top: Optional[int] = None,
) -> None:
    """Только фишки на доске, выброшенные и кубики (без подсказок)."""
    self.canvas.delete(TAG_PIECES)
    assert self.state is not None
    board = self.state.board
    h_tri = self._triangle_height()

    for i in range(POINTS_COUNT):
        cx, cy, direction = self.get_coords(i)
        cnt, pcol = board[i]
        if cnt > 0:
            step = self._stack_step(cnt)
            for j in range(cnt):
                if hide_source_top is not None and i == hide_source_top and j == cnt - 1:
                    continue
                py = cy + (self.piece_r + 5 + j * step) * direction
                sel = self.selected_point == i and j == cnt - 1
                self.draw_piece(cx, py, pcol, is_selected=sel, tags=TAG_PIECES)

    white_out_x, white_out_y = self.margin_x - self.out_pad, self.margin_y + 20
    black_out_x, black_out_y = (
        self.margin_x + self.board_w + (self.out_pad - 40),
        self.margin_y + self.board_h - 20 - self.board_h * 0.4,
    )

```

```

    )

    self.draw_out_checkers(white_out_x + 20, white_out_y + 10, self.state.out_count[WHITE],
WHITE, 1)

    self.draw_out_checkers(black_out_x + 20, black_out_y + self.board_h * 0.4 - 10,
self.state.out_count[BLACK], BLACK, -1)


    cur_dice = temp_dice if temp_dice is not None else self.state.dice

    self._draw_dice_on_shell(cur_dice)


def _draw_hints_layer(self) -> None:

    """Подсказки только при выбранной шашке; перед отрисовкой старые подсказки
снимаются."""

    self.canvas.delete(TAG_HINTS)

    if (
        not self.state
        or self.is_animating
        or self.selected_point is None
        or not self.valid_moves
    ):
        return


    board = self.state.board
    turn = self.state.current_turn
    white_out_x, white_out_y = self.margin_x - self.out_pad, self.margin_y + 20
    black_out_x, black_out_y = (
        self.margin_x + self.board_w + (self.out_pad - 40),
        self.margin_y + self.board_h - 20 - self.board_h * 0.4,
    )


    for target, dice_used in self.valid_moves.items():
        du = dice_used if isinstance(dice_used, list) else [dice_used]
        hint_sum = sum(du)
        label = "-".join(str(x) for x in du) if len(du) > 1 else str(hint_sum)
        if target == "out":

```

```

tx, ty = (
    (white_out_x + 20, white_out_y - 20)
    if turn == WHITE
    else (black_out_x + 20, black_out_y + self.board_h * 0.4 + 20)
)
self._draw_move_hint(tx, ty, label, 1)
else:
    tx, ty, td = self.get_coords(target)
    t_cnt, t_col = board[target]
    t_py = (
        ty + (self.piece_r + 5 + t_cnt * (self.piece_r * 2.2)) * td
        if t_col == turn
        else ty + (self.piece_r + 5) * td
    )
    self._draw_move_hint(tx, t_py, label, td)

```

```

def draw_board(self, full_redraw: bool = False, temp_dice: Optional[List[int]] = None):

```

```

    if not self.state:

```

```

        return

```

```

    if full_redraw or not self._shell_drawn:

```

```

        self._draw_board_shell()

```

```

        self._shell_drawn = True

```

```

    self.update_score_display()

```

```

    if not self.is_animating:

```

```

        self.canvas.delete(TAG_FLOATING)

```

```

    self._draw_pieces_layer(temp_dice=temp_dice, hide_source_top=None)

```

```

    self._draw_hints_layer()

```

```

    self._apply_z_order()

```

```

def _apply_z_order(self) -> None:

```

```

    """

```

```

    Порядок слоёв (снизу вверх):

```

```

    - фон/корпус (shell)

```


- шашки (pieces) и летящая (floating)
- центральная перегородка (center_bar)
- кубики (dice_draw), подсказки (hints), UI — всегда поверх всего

"""

Центр поверх шашек, но НЕ поверх кубиков/подсказок/UI

self.canvas.tag_raise(TAG_PIECES)

self.canvas.tag_raise(TAG_FLOATING)

self.canvas.tag_raise(TAG_CENTER_BAR)

self.canvas.tag_raise("dice_draw")

self.canvas.tag_raise(TAG_HINTS)

self.canvas.tag_raise("ui")

def _draw_move_hint(self, cx, piece_y, label: str, direction):

rw, rh = 32, 24

tri_h = 18

offset = (tri_h + rh) * (-direction)

rect_y = piece_y - offset

self.canvas.create_rectangle(

cx - rw // 2,

rect_y - rh // 2,

cx + rw // 2,

rect_y + rh // 2,

fill="#7bed9f",

outline="white",

width=2,

tags=TAG_HINTS,

)

self.canvas.create_text(cx, rect_y, text=label, fill="black", font=("Arial", 11, "bold"),
tags=TAG_HINTS)

base_y = rect_y + (rh // 2) * (-direction)

apex_y = piece_y

self.canvas.create_polygon(cx, apex_y, cx - 12, base_y, cx + 12, base_y, fill="#7bed9f",
outline="white", width=2, tags=TAG_HINTS)

```

def calc_moves(self, start: int) -> Dict:
    if not self.state:
        return {}
    return legal_moves_maximal_first(self.state, start)

def _move_endpoints(self, start: int, end: Union[int, str]) -> Tuple[float, float, float, float, str]:
    """Экранные координаты верхней шашки на start и целевой точки (до изменения
state)."""
    assert self.state is not None
    color = self.state.current_turn
    cnt = self.state.board[start][0]
    sx, sy = self._stack_top_xy(start, cnt)

    if end == "out":
        if color == WHITE:
            ex, ey = (self.margin_x - self.out_pad + 20), self.margin_y + 40
        else:
            ex, ey = (self.margin_x + self.board_w + (self.out_pad - 20)), self.margin_y +
self.board_h - 40
        else:
            ex, ey, ed = self.get_coords(end)
            cnt_end = self.state.board[end][0]
            if self.state.board[end][1] == color:
                # после хода стопка станет (cnt_end + 1) — используем её шаг укладки
                step_new = self._stack_step(cnt_end + 1)
                ey = ey + (self.piece_r + 5 + cnt_end * step_new) * ed
            else:
                ey = ey + (self.piece_r + 5) * ed
            return sx, sy, float(ex), float(ey), color

def animate_move(self, start: int, end: Union[int, str], dice_used: List[int]) -> None:
    """
    Плавный линейный перенос шашки: верхняя с start скрыта в слое pieces,

```

движущаяся отрисовывается только с тегом `floating`; каждый кадр обновляется только `floating`.

```
"""
```

```
self.is_animating = True
```

```
du = dice_used if isinstance(dice_used, list) else [dice_used]
```

```
self.selected_point = None
```

```
self.valid_moves = {}
```

```
self.canvas.delete(TAG_HINTS)
```

```
sx, sy, ex, ey, col = self._move_endpoints(start, end)
```

```
self._anim_pending = (start, end, du)
```

```
self._anim_sx, self._anim_sy = sx, sy
```

```
self._anim_ex, self._anim_ey = ex, ey
```

```
self._anim_color = col
```

```
self._anim_frame = 0
```

```
self._anim_frames = ANIM_MOVE_FRAMES
```

```
# Параметры траектории: мягкое смещение (очень небольшое), без «падения с неба».
```

```
center_y = self.margin_y + self.board_h / 2
```

```
self._anim_arc_dir = -1 if sy > center_y else 1
```

```
path = _path_for(col)
```

```
self._anim_from_head = path.index(start) == 0
```

```
mid_x = self.margin_x + self.board_w / 2
```

```
# Для проверок пересечения берём максимально широкую половину
```

```
half_left = getattr(self, "center_bar_left_half", self.center_bar_w / 2)
```

```
half_right = getattr(self, "center_bar_right_half", self.center_bar_w / 2)
```

```
bar_half = max(half_left, half_right) + (self.piece_r + 6)
```

```
bar_l, bar_r = mid_x - bar_half, mid_x + bar_half
```

```
pad = self.piece_r * 1.8 # больше зазор, чтобы не задевало перегородку
```

```
pts: List[Tuple[float, float]] = [(sx, sy)]
```

```
self._anim_detour_side: Optional[int] = None # -1 слева от bar, +1 справа, None без обхода
```

```

if end != "out":
    crosses_bar = (sx < bar_l and ex > bar_r) or (sx > bar_r and ex < bar_l)
    if crosses_bar:
        side_x = (bar_l - pad) if sx < bar_l else (bar_r + pad)
        self._anim_detour_side = -1 if sx < bar_l else 1

        # Переход на другую сторону: строим один гладкий путь (Bezier) через точку у
перегородки
        # и в центре поля. Так не будет “остановки/перезапуска” на поворотах.
        p0 = (sx, sy)
        p1 = (side_x, center_y)
        p2 = (ex, ey)
        self._anim_pts = [p0, p1, p2]
        self._anim_poly = self._quadratic_bezier_polyline(p0, p1, p2, samples=max(40,
self._anim_frames))
        self.canvas.delete(TAG_FLOATING)
        self._draw_pieces_layer(temp_dice=None, hide_source_top=start)
        self._anim_step_linear()
        return
    pts.append((ex, ey))
    self._anim_pts = pts
    self._anim_poly = self._catmull_rom_polyline(pts, samples_per_seg=20)

    self.canvas.delete(TAG_FLOATING)
    self._draw_pieces_layer(temp_dice=None, hide_source_top=start)
    self._anim_step_linear()

def _anim_step_linear(self) -> None:
    total = self._anim_frames
    f = self._anim_frame
    if f > total:
        self.canvas.delete(TAG_FLOATING)
        pending = self._anim_pending
        self._anim_pending = None
        if pending is not None:

```

```

        s, e, d = pending
        self.complete_move(s, e, d)
    return

# На последнем кадре — строго в цель и без лишнего кадра/рывка
if f >= total:
    x, y = self._anim_ex, self._anim_ey
    self.canvas.delete(TAG_FLOATING)
    self.draw_piece(x, y, self._anim_color, is_selected=False, tags=TAG_FLOATING)
    # завершить ход сразу (без дополнительного after)
    self._anim_frame = total + 1
    self.root.after(0, self._anim_step_linear)
    return

t = (f / total) if total else 1.0
t = max(0.0, min(1.0, t))

# smoothstep easing (без перелёта)
te = t * t * (3 - 2 * t)

pts = getattr(self, "_anim_poly", None)
if not pts or len(pts) < 2:
    pts = [(self._anim_sx, self._anim_sy), (self._anim_ex, self._anim_ey)]

# Проход по ломаной по длине (чтобы не было рывков на стыках сегментов)
seg_lengths: List[float] = []
total_len = 0.0
for i in range(len(pts) - 1):
    x0, y0 = pts[i]
    x1, y1 = pts[i + 1]
    ln = ((x1 - x0) ** 2 + (y1 - y0) ** 2) ** 0.5
    seg_lengths.append(ln)
    total_len += ln

```

```

if total_len <= 1e-6:
    x, y = pts[-1]
    seg_i = 0
    segs = 1
else:
    dist = te * total_len
    acc = 0.0
    seg_i = 0
    for i, ln in enumerate(seg_lengths):
        if dist <= acc + ln or i == len(seg_lengths) - 1:
            seg_i = i
            break
        acc += ln
    ln = seg_lengths[seg_i]
    local = 0.0 if ln <= 1e-6 else (dist - acc) / ln
    x0, y0 = pts[seg_i]
    x1, y1 = pts[seg_i + 1]
    x = x0 + (x1 - x0) * local
    y = y0 + (y1 - y0) * local
    segs = len(seg_lengths)

```

Очень лёгкое «человеческое» смещение (без ощущения, что шашку бросили сверху).

```

lift = math.sin(math.pi * t)
base = 0.025 if getattr(self, "_anim_from_head", False) else 0.045
arc_h = self.sh * base

```

При обходе перегородки дополнительную дугу почти убираем — кривая уже достаточно мягкая.

```

arc_scale = 0.0 if getattr(self, "_anim_detour_side", None) is not None else 1.0
y += getattr(self, "_anim_arc_dir", 1) * arc_h * lift * arc_scale
self.canvas.delete(TAG_FLOATING)
self.draw_piece(x, y, self._anim_color, is_selected=False, tags=TAG_FLOATING)
self._apply_z_order()
self._anim_frame += 1
self.root.after(ANIM_FPS_MS, self._anim_step_linear)

```

```

def complete_move(self, start, end, dice_used):
    dice_used = dice_used if isinstance(dice_used, list) else [dice_used]
    assert self.state is not None

    self.state = apply_move(self.state, start, end, dice_used)

    self.selected_point = None
    self.is_animating = False

    if self.state.out_count[self.state.current_turn] == CHECKERS_PER_PLAYER:
        self.show_victory_screen(self.state.current_turn)
        return

    has_moves = player_has_any_legal_move(self.state)

    if not self.state.dice or not has_moves:
        self.next_turn()
    else:
        self.draw_board(full_redraw=False)
        self.trigger_bot_if_needed()

def next_turn(self):
    assert self.state is not None
    self.state.first_turn[self.state.current_turn] = False
    self.state.current_turn = BLACK if self.state.current_turn == WHITE else WHITE
    self.state.head_taken_this_turn = 0
    self.selected_point = None
    self.valid_moves = {}
    self.draw_board(full_redraw=False)
    self.root.after(500, self.animate_dice)

def animate_dice(self):

```

```

self.waiting_for_dice = True
for i in range(10):
    self.root.after(i * 50, lambda s=i: self._dice_roll_step(s))

def _dice_roll_step(self, step):
    if not self.game_active or self.state is None:
        return
    d = [random.randint(1, 6), random.randint(1, 6)]
    self.draw_board(full_redraw=False, temp_dice=d)
    if step == 9:
        self.state.is_double = d[0] == d[1]
        self.state.dice = [d[0], d[1], d[0], d[1]] if self.state.is_double else d
        self.waiting_for_dice = False
        self.draw_board(full_redraw=False)

        if not player_has_any_legal_move(self.state):
            self.root.after(1000, self.next_turn)
        else:
            self.trigger_bot_if_needed()

def trigger_bot_if_needed(self):
    if self.bot_level and self.state and self.state.current_turn != self.human_color:
        self.root.after(500, self.bot_turn_step)

def bot_turn_step(self):
    if not self.game_active or self.is_animating or self.waiting_for_dice or not self.state or not self.state.dice:
        return
    move = choose_bot_move(self.state, self.bot_level)
    if move is None:
        self.next_turn()
        return
    start, target, dice_used = move
    self.animate_move(start, target, dice_used)

```



```

def on_click(self, event):
    if (
        self.is_animating
        or self.waiting_for_dice
        or not self.game_active
        or not self.state
        or (self.bot_level and self.state.current_turn != self.human_color)
    ):
        return

    white_out_x, black_out_x = self.margin_x - self.out_pad, self.margin_x + self.board_w +
    (self.out_pad - 40)

    if self.selected_point is not None and "out" in self.valid_moves:
        if self.state.current_turn == WHITE and white_out_x <= event.x <= white_out_x + 60 and
        self.margin_y <= event.y <= self.margin_y + self.board_h:
            self.animate_move(self.selected_point, "out", self.valid_moves["out"])
            return

        elif self.state.current_turn == BLACK and black_out_x <= event.x <= black_out_x + 60
        and self.margin_y <= event.y <= self.margin_y + self.board_h:
            self.animate_move(self.selected_point, "out", self.valid_moves["out"])
            return

    for i in range(POINTS_COUNT):
        cx, cy, direction = self.get_coords(i)
        if cx - self.point_w // 2 <= event.x <= cx + self.point_w // 2:
            if (direction == -1 and event.y > self.margin_y + self.board_h / 2) or (
                direction == 1 and event.y < self.margin_y + self.board_h / 2
            ):
                if self.selected_point is not None and i in self.valid_moves:
                    self.animate_move(self.selected_point, i, self.valid_moves[i])
                    return

                elif self.state.board[i][1] == self.state.current_turn:

```

```
self.selected_point = i
self.valid_moves = self.calc_moves(i)
self.draw_board(full_redraw=False)
return
```

```
self.selected_point = None
self.valid_moves = {}
self.draw_board(full_redraw=False)
```

```
def confirm_exit(self):
    if not self.game_active:
        self.show_menu()
        return
    win = tk.Toplevel(self.root)
    win.title("Выход из игры")
    win.geometry("400x180")
    win.configure(bg="#1a110a")
    win.transient(self.root)
    win.grab_set()

    tk.Label(win, text="Сдаться и выйти в меню?", font=self.font_ui, bg="#1a110a",
fg="white").pack(pady=(25, 15))

    tk.Label(win, text="Противник будет объявлен победителем.", font=("Verdana", 10),
bg="#1a110a", fg="#aaa").pack(pady=5)

    f = tk.Frame(win, bg="#1a110a")
    f.pack(pady=20)

    def do_surrender():
        win.destroy()

        surrendering = self.human_color if self.bot_level else self.state.current_turn if self.state
else WHITE

        winner = BLACK if surrendering == WHITE else WHITE

        self.show_victory_screen(winner, surrendered=True)

    tk.Button(f, text="Да, сдаться", font=self.font_ui, bg="#5e1914", fg="white",
command=do_surrender).pack(side="left", padx=10)
```

```
tk.Button(f, text="Отмена", font=self.font_ui, bg="#3e2716", fg=self.clr_accent,
command=win.destroy).pack(side="left", padx=10)
```

```
def show_victory_screen(self, winner, surrendered=False):
    self.game_active = False
    if self.timer_id:
        self.root.after_cancel(self.timer_id)
    elapsed = int(time.time() - self.start_game_time)
    mins, secs = elapsed // 60, elapsed % 60
    win_text = "БЕЛЫЕ ПОБЕДИЛИ!" if winner == WHITE else "ЧЕРНЫЕ ПОБЕДИЛИ!"
    if surrendered:
        win_text += " (Сдача)"
    win = tk.Toplevel(self.root)
    win.title("Конец игры")
    w, h = int(self.board_w), int(self.board_h)
    x = (self.sw - w) // 2
    y = (self.sh - h) // 2
    win.geometry(f"{w}x{h}+{x}+{y}")
    win.configure(bg="#1a110a")
    win.transient(self.root)
    win.grab_set()
    big_font = ("Garamond", max(28, int(self.sh * 0.04)), "bold")
    btn_font = ("Verdana", max(14, int(self.sh * 0.02)), "bold")
    tk.Label(win, text=win_text, font=big_font, bg="#1a110a", fg="#00ff00").pack(pady=(h // 6,
h // 12))
    tk.Label(win, text=f"Время партии: {mins:02d}:{secs:02d}", font=self.font_ui,
bg="#1a110a", fg="white").pack(pady=15)
    tk.Button(
        win,
        text="В ГЛАВНОЕ МЕНЮ",
        font=btn_font,
        bg="#d4af37",
        fg="black",
        padx=40,
```

```
    pady=15,  
    command=lambda: [win.destroy(), self.show_menu()],  
).pack(pady=h // 6)
```

```
if __name__ == "__main__":  
    root = tk.Tk()  
    app = LongNardMaster(root)  
    root.mainloop()
```